

Análisis estadístico y predictivo de la Premier League 2022-2023

Brais Muíño Otero

2026-06-13

Índice

A continuación se presenta la estructura del proyecto:

1. Introducción

- 1.1. Contexto del problema
- 1.2. Objetivos del trabajo
- 1.3. Descripción del dataset

2. Carga y preparación de los datos

- 2.1. Importación del archivo
- 2.2. Inspección inicial del dataset
- 2.3. Comprobación de valores perdidos
- 2.4. Conversión de variables
- 2.5. Creación de variables derivadas

3. ¿Cómo se juega en la Premier League?

- 3.1. Distribución de goles
- 3.2. Distribución de resultados
- 3.3. Comparación entre equipo local y visitante
- 3.4. Relación entre variables ofensivas y goles
- 3.5. Correlaciones entre variables numéricas

4. ¿Existe realmente la ventaja de jugar en casa?

- 4.1. Intervalo de confianza para la media de goles locales
- 4.2. Contraste de medias: goles local vs visitante
- 4.3. Contraste de medias: tiros a puerta local vs visitante
- 4.4. Asociación entre tarjetas rojas y resultado

5. ¿Cuánto de bueno es cada equipo realmente?

- 5.1. Modelo Dixon-Coles: estimación por máxima verosimilitud
- 5.2. Ranking de fuerza de ataque y debilidad defensiva
- 5.3. Comparación con la clasificación real

6. ¿Quién habría ganado la liga en otras circunstancias?

- 6.1. Simulación Monte Carlo de la temporada (10.000 iteraciones)
- 6.2. Probabilidades de campeón, top 4 y descenso
- 6.3. Separando calidad de fortuna

7. ¿Se puede predecir el resultado de un partido?

- 7.1. Preparación y división de datos
- 7.2. Regresión de Poisson: predicción de goles
 - 7.2.1. Modelo de Poisson y medidas de bondad de ajuste
 - 7.2.2. Ejemplos de predicción de goles
 - 7.2.3. Diagnóstico del modelo y validación por supuestos
- 7.3. Regresión logística binomial: predicción de victoria local
 - 7.3.1. Modelo logístico y medidas de bondad de ajuste
 - 7.3.2. Ejemplos de predicción e interpretación de Odds Ratios

- 7.3.3. Diagnóstico del modelo y validación por supuestos
- 7.4. Análisis discriminante lineal (LDA)
- 7.5. Clasificación mediante KNN
- 7.6. Comparación de modelos y validación cruzada
- 8. ¿Qué patrones de juego existen entre equipos?**
 - 8.1. Reducción de dimensionalidad mediante PCA
 - 8.2. Identificación de grupos mediante K-means
 - 8.3. Interpretación táctica de los clusters
- 9. ¿Cómo optimizar el rendimiento de un equipo?**
 - 9.1. Planteamiento del problema y función objetivo
 - 9.2. Restricciones y resolución del modelo
 - 9.3. Análisis de sensibilidad
 - 9.4. Interpretación de resultados
- 10. Conclusiones**
 - 10.1. Principales hallazgos
 - 10.2. Limitaciones
 - 10.3. Líneas futuras

1. Introducción

1.1. Contexto del problema

El fútbol es uno de los deportes con mayor impacto a nivel mundial, y en los últimos años el análisis de datos ha adquirido un papel fundamental en la comprensión del juego. Gracias al desarrollo de herramientas estadísticas y técnicas de aprendizaje automático, es posible analizar el rendimiento de los equipos, identificar patrones de juego y estudiar los factores que influyen en el resultado de los partidos.

En este contexto, el análisis de datos deportivos permite transformar información cuantitativa en conocimiento útil, tanto para la toma de decisiones como para la interpretación del juego desde una perspectiva más objetiva. Variables como el número de goles, tiros, tiros a puerta, corners o tarjetas ofrecen una descripción detallada del desarrollo de cada partido.

El presente trabajo se centra en el análisis de una temporada de la Premier League inglesa, una de las competiciones más importantes del fútbol internacional, utilizando técnicas de estadística y ciencia de datos.

1.2. Objetivos del trabajo

El objetivo principal de este estudio es analizar estadísticamente los partidos de la temporada 2022-2023 de la Premier League, así como aplicar diferentes técnicas de modelización para comprender mejor los factores que influyen en el resultado de los encuentros. De forma más concreta, los objetivos del trabajo son:

- Realizar un análisis exploratorio de las variables más relevantes del dataset.
- Estudiar la distribución de los goles y los resultados de los partidos.
- Analizar la relación entre variables ofensivas y el rendimiento de los equipos.
- Aplicar técnicas de inferencia estadística para validar hipótesis.
- Estimar la fuerza de ataque y debilidad defensiva de cada equipo mediante el modelo Dixon-Coles.
- Simular la temporada mediante métodos de Monte Carlo para cuantificar el papel de la suerte en la clasificación final.
- Aplicar métodos de clasificación supervisada, como regresión logística, LDA y KNN, para predecir resultados.
- Utilizar métodos de reducción de dimensionalidad, como el análisis de componentes principales (PCA).
- Identificar patrones de juego mediante técnicas de clustering (K-means).
- Plantear un problema de optimización lineal en el contexto deportivo.

1.3. Descripción del dataset

El conjunto de datos utilizado en este trabajo corresponde a la temporada 2022-2023 de la Premier League inglesa. Cada fila del dataset representa un partido, mientras que las columnas contienen información sobre diferentes aspectos del mismo.

Entre las variables más relevantes se encuentran:

- FTHG: número de goles marcados por el equipo local.
- FTAG: número de goles marcados por el equipo visitante.
- FTR: resultado final del partido (H = victoria local, D = empate, A = victoria visitante).
- HS / AS: número total de tiros realizados por cada equipo (H = home/local, A = away/visitante).
- HST / AST: número de tiros a puerta.
- HC / AC: número de saques de esquina.
- HF / AF: número de faltas cometidas.
- HY / AY: número de tarjetas amarillas.
- HR / AR: número de tarjetas rojas.

Además, el dataset incluye información adicional como el árbitro del partido y estadísticas de la primera parte. A partir de estas variables, se pueden construir nuevas variables derivadas que permiten enriquecer el análisis y facilitar la interpretación de los resultados.

El tamaño del dataset, con aproximadamente 380 partidos, lo hace adecuado para aplicar técnicas de análisis estadístico y aprendizaje automático sin que el volumen de datos resulte excesivo.

2. Carga y preparación de los datos

2.1. Importación del archivo

En esta sección se importan las librerías necesarias para todo el análisis y se carga el conjunto de datos de la temporada 2022-2023 de la Premier League inglesa. Centralizar las dependencias al inicio favorece la reproducibilidad del estudio y evita cargas redundantes a lo largo del documento.

```
# Librerías utilizadas a lo largo de todo el análisis
library(corrplot)      # Visualización de matrices de correlación
library(lmtest)        # Contrastes sobre modelos (razón de verosimilitud)
library(psc1)          # Pseudo-R2 (McFadden) para modelos GLM
library(MASS)          # Análisis discriminante lineal y cuadrático (LDA/QDA)
library(class)         # Clasificación mediante KNN
library(pROC)          # Curvas ROC y AUC
library(cluster)       # Métricas de agrupamiento (silueta)
library(lpSolveAPI)    # Programación lineal y análisis de sensibilidad

# Carga del archivo CSV del dataset
datos <- read.csv("epl_results_2022-23.csv")
head(datos, 10) # Primeras 10 filas del dataset
```

##	Date	Time	HomeTeam	AwayTeam	FTHG	FTAG	FTR	HTHG	HTAG	HTR
## 1	05/08/2022	20:00	Crystal Palace	Arsenal	0	2	A	0	1	A
## 2	06/08/2022	12:30	Fulham	Liverpool	2	2	D	1	0	H
## 3	06/08/2022	15:00	Bournemouth	Aston Villa	2	0	H	1	0	H
## 4	06/08/2022	15:00	Leeds	Wolves	2	1	H	1	1	D

```
## 5 06/08/2022 15:00 Newcastle Nottingham 2 0 H 0 0 D
## 6 06/08/2022 15:00 Tottenham Southampton 4 1 H 2 1 H
## 7 06/08/2022 17:30 Everton Chelsea 0 1 A 0 1 A
## 8 07/08/2022 14:00 Leicester Brentford 2 2 D 1 0 H
## 9 07/08/2022 14:00 Man United Brighton 1 2 A 0 2 A
## 10 07/08/2022 16:30 West Ham Man City 0 2 A 0 1 A
## Referee HS AS HST AST HF AF HC AC HY AY HR AR
## 1 A Taylor 10 10 2 2 16 11 3 5 1 2 0 0
## 2 A Madley 9 11 3 4 7 9 4 4 2 0 0 0
## 3 P Bankes 7 15 3 2 18 16 5 5 3 3 0 0
## 4 R Jones 12 15 4 6 13 9 6 4 2 0 0 0
## 5 S Hooper 23 5 10 0 9 14 11 1 0 3 0 0
## 6 A Marriner 18 10 8 2 11 6 10 2 3 0 0 0
## 7 C Pawson 8 15 4 6 14 11 4 16 3 2 0 0
## 8 J Gillett 14 8 5 3 6 5 5 6 0 0 0 0
## 9 P Tierney 17 15 5 4 7 12 6 2 4 1 0 0
## 10 M Oliver 6 14 1 2 8 4 1 4 0 1 0 0
```

La visualización de las primeras diez filas confirma la estructura “un partido por fila” y permite reconocer los tres bloques de información del dataset. El primero es contextual (Date, Time, HomeTeam, AwayTeam); el segundo recoge el desenlace deportivo (FTHG, FTAG, FTR y sus equivalentes al descanso HTHG, HTAG, HTR); y el tercero contiene las estadísticas de juego junto al árbitro (Referee): tiros (HS, AS), tiros a puerta (HST, AST), faltas (HF, AF), corners (HC, AC) y tarjetas (HY, AY, HR, AR). Así, la primera fila es Crystal Palace 0-2 Arsenal (FTR = A, victoria visitante) y la sexta Tottenham 4-1 Southampton (FTR = H, victoria local). Esta inspección verifica que los datos se han cargado correctamente, que las columnas están bien alineadas y que los tipos aparentes (texto para equipos y resultados, enteros para los conteos) son los esperados.

2.2. Inspección inicial del dataset

Se realiza una exploración inicial del dataset con el objetivo de conocer su estructura, número de observaciones, variables disponibles y tipos de datos.

```
dim(datos) # Dimensión NxM de los datos (N filas, M columnas)
```

```
## [1] 380 23
```

Aquí se puede observar la dimensión del dataset: 380 filas (una por cada partido de la temporada) y 23 columnas (cada una corresponde a una característica de un partido, a una variable).

```
colnames(datos) # Nombres de las columnas del dataset
```

```
## [1] "Date"      "Time"      "HomeTeam"  "AwayTeam"  "FTHG"      "FTAG"
## [7] "FTR"       "HTHG"      "HTAG"      "HTR"       "Referee"    "HS"
## [13] "AS"        "HST"       "AST"       "HF"        "AF"         "HC"
## [19] "AC"        "HY"        "AY"        "HR"        "AR"
```

El dataset los nombres de todas las variables que contiene el dataset.

```
str(datos) # Estructura del dataset y tipo de variables
```

```
## 'data.frame': 380 obs. of 23 variables:
## $ Date : chr "05/08/2022" "06/08/2022" "06/08/2022" "06/08/2022" ...
## $ Time : chr "20:00" "12:30" "15:00" "15:00" ...
## $ HomeTeam: chr "Crystal Palace" "Fulham" "Bournemouth" "Leeds" ...
## $ AwayTeam: chr "Arsenal" "Liverpool" "Aston Villa" "Wolves" ...
## $ FTHG : int 0 2 2 2 2 4 0 2 1 0 ...
## $ FTAG : int 2 2 0 1 0 1 1 2 2 2 ...
## $ FTR : chr "A" "D" "H" "H" ...
## $ HTHG : int 0 1 1 1 0 2 0 1 0 0 ...
## $ HTAG : int 1 0 0 1 0 1 1 0 2 1 ...
## $ HTR : chr "A" "H" "H" "D" ...
## $ Referee : chr "A Taylor" "A Madley" "P Bankes" "R Jones" ...
## $ HS : int 10 9 7 12 23 18 8 14 17 6 ...
## $ AS : int 10 11 15 15 5 10 15 8 15 14 ...
## $ HST : int 2 3 3 4 10 8 4 5 5 1 ...
## $ AST : int 2 4 2 6 0 2 6 3 4 2 ...
## $ HF : int 16 7 18 13 9 11 14 6 7 8 ...
## $ AF : int 11 9 16 9 14 6 11 5 12 4 ...
## $ HC : int 3 4 5 6 11 10 4 5 6 1 ...
## $ AC : int 5 4 5 4 1 2 16 6 2 4 ...
## $ HY : int 1 2 3 2 0 3 3 0 4 0 ...
## $ AY : int 2 0 3 0 3 0 2 0 1 1 ...
## $ HR : int 0 0 0 0 0 0 0 0 0 0 ...
## $ AR : int 0 0 0 0 0 0 0 0 0 0 ...
```

Estructura de los datos, tipos de variables (categóricas, fecha, numéricas)

```
summary(datos) # Resumen de los datos (mínimo, máximo, mediana, media y cuartiles)
```

```
##      Date           Time           HomeTeam       AwayTeam
## Length:380      Length:380      Length:380      Length:380
## Class :character Class :character Class :character Class :character
## Mode :character Mode :character Mode :character Mode :character
##
##
##
##      FTHG           FTAG           FTR           HTHG
## Min.   :0.000      Min.   :0.000      Length:380      Min.   :0.0000
## 1st Qu.:1.000      1st Qu.:0.000      Class :character 1st Qu.:0.0000
## Median :1.000      Median :1.000      Mode :character   Median :1.0000
## Mean   :1.634      Mean   :1.218                      Mean   :0.7579
## 3rd Qu.:2.000      3rd Qu.:2.000                      3rd Qu.:1.0000
## Max.   :9.000      Max.   :6.000                      Max.   :5.0000
##      HTAG           HTR           Referee           HS
## Min.   :0.0000      Length:380      Length:380      Min.   : 1.00
## 1st Qu.:0.0000      Class :character Class :character 1st Qu.:10.00
## Median :0.0000      Mode :character Mode :character   Median :14.00
## Mean   :0.5632                      Mean   :13.95
## 3rd Qu.:1.0000                      3rd Qu.:17.00
## Max.   :3.0000                      Max.   :33.00
```

##	AS	HST	AST	HF
##	Min. : 1.00	Min. : 0.000	Min. : 0.000	Min. : 3.0
##	1st Qu.: 8.00	1st Qu.: 3.000	1st Qu.: 2.000	1st Qu.: 8.0
##	Median :11.00	Median : 5.000	Median : 4.000	Median :10.0
##	Mean :11.31	Mean : 4.908	Mean : 3.895	Mean :10.6
##	3rd Qu.:15.00	3rd Qu.: 7.000	3rd Qu.: 5.000	3rd Qu.:13.0
##	Max. :30.00	Max. :15.000	Max. :11.000	Max. :23.0
##	AF	HC	AC	HY
##	Min. : 3.00	Min. : 0.000	Min. : 0.000	Min. :0.000
##	1st Qu.: 8.00	1st Qu.: 3.000	1st Qu.: 2.000	1st Qu.:1.000
##	Median :11.00	Median : 5.000	Median : 4.000	Median :2.000
##	Mean :10.93	Mean : 5.637	Mean : 4.471	Mean :1.671
##	3rd Qu.:13.00	3rd Qu.: 8.000	3rd Qu.: 6.000	3rd Qu.:3.000
##	Max. :24.00	Max. :17.000	Max. :19.000	Max. :6.000
##	AY	HR	AR	
##	Min. :0.000	Min. :0.00000	Min. :0.00000	
##	1st Qu.:1.000	1st Qu.:0.00000	1st Qu.:0.00000	
##	Median :2.000	Median :0.00000	Median :0.00000	
##	Mean :1.916	Mean :0.04737	Mean :0.02632	
##	3rd Qu.:3.000	3rd Qu.:0.00000	3rd Qu.:0.00000	
##	Max. :7.000	Max. :2.00000	Max. :1.00000	

Resumen de los datos (mínimo, máximo, mediana, media y cuartiles)

A partir de la inspección inicial del dataset, se observa que el conjunto de datos está compuesto por 380 observaciones y 23 variables, lo que corresponde al número total de partidos disputados en una temporada completa de la Premier League.

Las variables incluyen información tanto categórica como numérica. En particular, variables como HomeTeam, AwayTeam, Referee y FTR son de tipo categórico (aunque inicialmente aparecen como carácter), mientras que variables como FTHG, FTAG, HS, HST, HC o HY son de tipo numérico y representan estadísticas del partido.

El dataset recoge información detallada de cada encuentro, incluyendo goles, tiros, tiros a puerta, faltas, corners y tarjetas, lo que lo hace adecuado para realizar un análisis estadístico completo.

El resumen estadístico de las variables numéricas permite extraer varias conclusiones relevantes.

En primer lugar, el número medio de goles del equipo local (1.63) es superior al del equipo visitante (1.22), lo que sugiere la existencia de ventaja de jugar en casa.

Asimismo, se observa que los equipos locales realizan, en promedio, más tiros (13.95 frente a 11.31) y más tiros a puerta (4.91 frente a 3.90), lo que refuerza la idea de un mayor dominio ofensivo del equipo local.

En cuanto a las faltas y tarjetas, ambos equipos presentan valores similares, aunque los equipos visitantes reciben ligeramente más tarjetas amarillas en promedio.

Por último, las tarjetas rojas son poco frecuentes, con valores medios cercanos a cero, lo que indica que son eventos poco habituales en los partidos.

2.3. Comprobación de valores perdidos

En esta sección se analiza la presencia de valores perdidos en el conjunto de datos, ya que estos pueden afectar al análisis estadístico y a la construcción de modelos.

La detección y tratamiento adecuado de los valores nulos (missing values) es un paso fundamental en cualquier proceso de análisis de datos, garantizando la consistencia y fiabilidad de los resultados.

```
sum(is.na(datos)) # Comprobación de valores perdidos en todo el dataset
```

```
## [1] 0
```

```
colSums(is.na(datos)) # Comprobación de valores perdidos por variable
```

```
##      Date      Time HomeTeam AwayTeam      FTHG      FTAG      FTR      HTHG
##      0         0         0         0         0         0         0         0
##      HTAG      HTR  Referee      HS      AS      HST      AST      HF
##      0         0         0         0         0         0         0         0
##      AF        HC        AC      HY      AY      HR      AR
##      0         0         0         0         0         0         0
```

No se detectan valores perdidos (nulos) en ninguna de las variables, por lo que no es necesario realizar ningún tratamiento adicional, facilitando positivamente el análisis posterior.

2.4. Conversión de variables

En esta sección se realiza la conversión de las variables del dataset a los tipos de datos adecuados, con el objetivo de garantizar un correcto análisis posterior.

En particular, se transforman las variables categóricas a factores y la variable de fecha a un formato apropiado, lo que permite su correcta utilización en métodos estadísticos y modelos de aprendizaje automático.

```
# Conversión de variables categóricas a factor
```

```
datos$HomeTeam <- as.factor(datos$HomeTeam)
```

```
datos$AwayTeam <- as.factor(datos$AwayTeam)
```

```
datos$Referee <- as.factor(datos$Referee)
```

```
datos$FTR <- as.factor(datos$FTR)
```

```
datos$HTR <- as.factor(datos$HTR)
```

```
datos$Date <- as.Date(datos$Date, format = "%d/%m/%Y") # Conversión de
# la variable Date a formato fecha
```

```
str(datos) # Estructura del dataset una vez realizada la conversión de variables
```

```
## 'data.frame':  380 obs. of  23 variables:
## $ Date      : Date, format: "2022-08-05" "2022-08-06" ...
## $ Time      : chr  "20:00" "12:30" "15:00" "15:00" ...
## $ HomeTeam: Factor w/ 20 levels "Arsenal","Aston Villa",...: 7 9 3 10 15 18 8 11 14 19 ...
## $ AwayTeam: Factor w/ 20 levels "Arsenal","Aston Villa",...: 1 12 2 20 16 17 6 4 5 13 ...
## $ FTHG     : int  0 2 2 2 2 4 0 2 1 0 ...
## $ FTAG     : int  2 2 0 1 0 1 1 2 2 2 ...
## $ FTR      : Factor w/ 3 levels "A","D","H": 1 2 3 3 3 3 1 2 1 1 ...
## $ HTHG     : int  0 1 1 1 0 2 0 1 0 0 ...
## $ HTAG     : int  1 0 0 1 0 1 1 0 2 1 ...
## $ HTR      : Factor w/ 3 levels "A","D","H": 1 3 3 2 2 3 1 3 1 1 ...
## $ Referee  : Factor w/ 25 levels "A Madley","A Marriner",...: 3 1 17 19 22 2 5 13 18 15 ...
## $ HS       : int  10 9 7 12 23 18 8 14 17 6 ...
## $ AS       : int  10 11 15 15 5 10 15 8 15 14 ...
## $ HST      : int  2 3 3 4 10 8 4 5 5 1 ...
```

```
## $ AST      : int  2 4 2 6 0 2 6 3 4 2 ...
## $ HF       : int  16 7 18 13 9 11 14 6 7 8 ...
## $ AF       : int  11 9 16 9 14 6 11 5 12 4 ...
## $ HC       : int  3 4 5 6 11 10 4 5 6 1 ...
## $ AC       : int  5 4 5 4 1 2 16 6 2 4 ...
## $ HY       : int  1 2 3 2 0 3 3 0 4 0 ...
## $ AY       : int  2 0 3 0 3 0 2 0 1 1 ...
## $ HR       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ AR       : int  0 0 0 0 0 0 0 0 0 0 ...
```

Las variables categóricas se han convertido a factores y la variable Date a formato fecha, lo que permite su correcto tratamiento en el análisis estadístico y en la construcción de modelos.

2.5. Creación de variables derivadas

En esta sección se crean nuevas variables a partir de las ya existentes con el objetivo de enriquecer el análisis y facilitar la construcción de modelos. Estas variables derivadas permiten capturar información relevante, como el número total de goles o la clasificación de los partidos según su resultado, mejorando así la capacidad explicativa del dataset.

```
datos$TotalGoals <- datos$FTHG + datos$FTAG # Num. total de goles en el partido
datos$HomeWin <- ifelse(datos$FTR == "H", 1, 0) # Variable binaria:
# victoria del equipo local
datos$Over2_5 <- ifelse(datos$TotalGoals > 2.5, 1, 0) # Variable binaria:
# más de 2.5 goles
datos$GoalDiff <- datos$FTHG - datos$FTAG # Diferencia de goles
datos$ShotsDiff <- datos$HS - datos$AS # Diferencia de tiros totales
datos$ShotsTargetDiff <- datos$HST - datos$AST # Diferencia de tiros a puerta
# Eficiencia ofensiva local (goles / tiros a puerta)
datos$EfficiencyHome <- ifelse(datos$HST > 0, datos$FTHG / datos$HST, 0)
# Eficiencia ofensiva visitante (goles / tiros a puerta)
datos$EfficiencyAway <- ifelse(datos$AST > 0, datos$FTAG / datos$AST, 0)
# Diferencia de eficiencias
datos$EfficiencyDiff <- datos$EfficiencyHome - datos$EfficiencyAway
# Precisión local (tiros a puerta / tiros totales)
datos$PrecisionHome <- ifelse(datos$HS > 0, datos$HST / datos$HS, 0)
# Precisión visitante (tiros a puerta / tiros totales)
datos$PrecisionAway <- ifelse(datos$AS > 0, datos$AST / datos$AS, 0)
# Total de tiros en el partido
datos$TotalShots <- datos$HS + datos$AS
# Ratio de tiros del equipo local (dominio ofensivo)
datos$ShotRatioHome <- ifelse((datos$HS + datos$AS) > 0,
                             datos$HS / (datos$HS + datos$AS), 0)
str(datos) # Estructura del dataset una vez creadas las variables derivadas
```

```
## 'data.frame': 380 obs. of 36 variables:
## $ Date      : Date, format: "2022-08-05" "2022-08-06" ...
## $ Time      : chr  "20:00" "12:30" "15:00" "15:00" ...
## $ HomeTeam  : Factor w/ 20 levels "Arsenal","Aston Villa",...: 7 9 3 10 15 18 8 11 14 19 ...
## $ AwayTeam  : Factor w/ 20 levels "Arsenal","Aston Villa",...: 1 12 2 20 16 17 6 4 5 13 ...
## $ FTHG      : int  0 2 2 2 2 4 0 2 1 0 ...
## $ FTAG      : int  2 2 0 1 0 1 1 2 2 2 ...
## $ FTR       : Factor w/ 3 levels "A","D","H": 1 2 3 3 3 3 1 2 1 1 ...
```

```

## $ HTHG      : int  0 1 1 1 0 2 0 1 0 0 ...
## $ HTAG      : int  1 0 0 1 0 1 1 0 2 1 ...
## $ HTR       : Factor w/ 3 levels "A","D","H": 1 3 3 2 2 3 1 3 1 1 ...
## $ Referee   : Factor w/ 25 levels "A Madley","A Marriner",...: 3 1 17 19 22 2 5 13 18 15 ...
## $ HS        : int  10 9 7 12 23 18 8 14 17 6 ...
## $ AS        : int  10 11 15 15 5 10 15 8 15 14 ...
## $ HST       : int  2 3 3 4 10 8 4 5 5 1 ...
## $ AST       : int  2 4 2 6 0 2 6 3 4 2 ...
## $ HF        : int  16 7 18 13 9 11 14 6 7 8 ...
## $ AF        : int  11 9 16 9 14 6 11 5 12 4 ...
## $ HC        : int  3 4 5 6 11 10 4 5 6 1 ...
## $ AC        : int  5 4 5 4 1 2 16 6 2 4 ...
## $ HY        : int  1 2 3 2 0 3 3 0 4 0 ...
## $ AY        : int  2 0 3 0 3 0 2 0 1 1 ...
## $ HR        : int  0 0 0 0 0 0 0 0 0 0 ...
## $ AR        : int  0 0 0 0 0 0 0 0 0 0 ...
## $ TotalGoals : int  2 4 2 3 2 5 1 4 3 2 ...
## $ HomeWin    : num  0 0 1 1 1 1 0 0 0 0 ...
## $ Over2_5    : num  0 1 0 1 0 1 0 1 1 0 ...
## $ GoalDiff   : int  -2 0 2 1 2 3 -1 0 -1 -2 ...
## $ ShotsDiff  : int  0 -2 -8 -3 18 8 -7 6 2 -8 ...
## $ ShotsTargetDiff: int  0 -1 1 -2 10 6 -2 2 1 -1 ...
## $ EfficiencyHome : num  0 0.667 0.667 0.5 0.2 ...
## $ EfficiencyAway : num  1 0.5 0 0.167 0 ...
## $ EfficiencyDiff : num  -1 0.167 0.667 0.333 0.2 ...
## $ PrecisionHome : num  0.2 0.333 0.429 0.333 0.435 ...
## $ PrecisionAway : num  0.2 0.364 0.133 0.4 0 ...
## $ TotalShots  : int  20 20 22 27 28 28 23 22 32 20 ...
## $ ShotRatioHome : num  0.5 0.45 0.318 0.444 0.821 ...

```

Tras la incorporación de las variables derivadas, el dataset presenta una estructura más completa y enriquecida, permitiendo una mejor representación del desarrollo de los partidos. Las nuevas variables introducen información adicional sobre el rendimiento ofensivo, la eficiencia y la diferencia entre equipos, aspectos que no se recogían de forma directa en las variables originales.

De este modo, el conjunto de datos resultante no solo amplía la información disponible, sino que también facilita la aplicación de técnicas de análisis más avanzadas, mejorando la capacidad interpretativa y predictiva del estudio.

3. ¿Cómo se juega en la Premier League?

En esta sección se lleva a cabo un análisis exploratorio de los datos con el objetivo de comprender las principales características del dataset y detectar patrones relevantes en el desarrollo de los partidos. A través de técnicas descriptivas y visualizaciones gráficas, se analizan variables clave relacionadas con los goles, los resultados y el rendimiento ofensivo de los equipos.

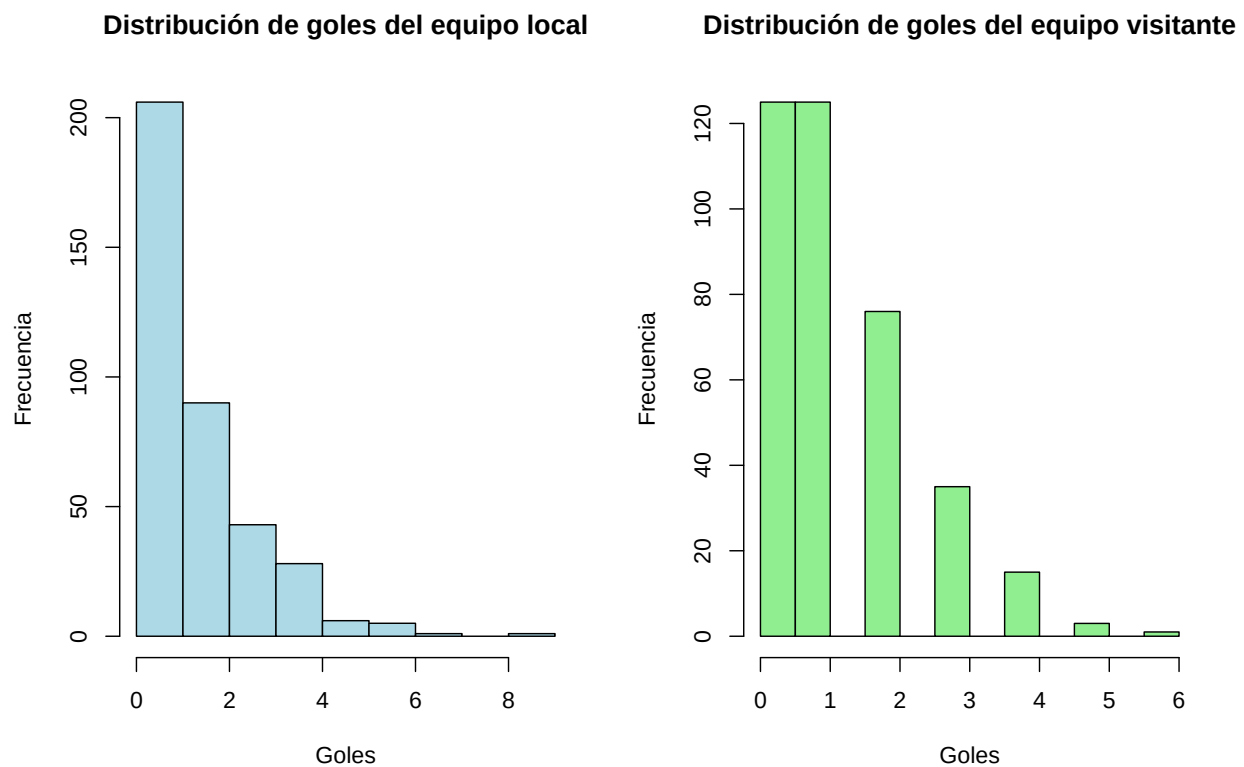
Este análisis permite identificar relaciones entre variables, tendencias generales y posibles factores que influyen en el resultado de los encuentros, sirviendo como base para las posteriores fases de modelización.

3.1. Distribución de goles

En esta sección se analiza la distribución de goles anotados por los equipos locales y visitantes, con el objetivo de comprender el comportamiento ofensivo de los partidos y detectar posibles patrones en la anotación de

goles.

```
# Histogramas de goles local y visitante (en rejilla 1x2)
par(mfrow = c(1, 2))
hist(datos$FTHG,
      main = "Distribución de goles del equipo local",
      xlab = "Goles", ylab = "Frecuencia",
      col = "lightblue")
hist(datos$FTAG,
      main = "Distribución de goles del equipo visitante",
      xlab = "Goles", ylab = "Frecuencia",
      col = "lightgreen")
```



```
par(mfrow = c(1, 1))
```

A partir de los histogramas, se observa que la mayoría de los partidos presentan un número reducido de goles, concentrándose principalmente entre 0 y 2. La distribución presenta una clara asimetría hacia la derecha, lo que indica la existencia de algunos partidos con un número elevado de goles, aunque estos son poco frecuentes.

Además, se aprecia que los equipos locales tienden a marcar ligeramente más goles que los visitantes, lo que refuerza la hipótesis de la ventaja de jugar en casa.

```
# Media de goles
mean(datos$FTHG) # Media goles local
```

```
## [1] 1.634211
```

```
mean(datos$FTAG) # Media goles visitante
```

```
## [1] 1.218421
```

Este resultado también se confirma al observar que la media de goles del equipo local es superior a la del equipo visitante.

En general, los partidos de la Premier League presentan un número moderado de goles, siendo poco frecuentes los encuentros con anotaciones muy elevadas. Asimismo, se confirma una ligera superioridad ofensiva de los equipos locales frente a los visitantes.

3.2. Distribución de resultados

En esta subsección se analiza la distribución de los resultados finales de los partidos (victoria local H, empate D y victoria visitante A) mediante su tabla de frecuencias, las proporciones correspondientes y un gráfico de barras, con el objetivo de cuantificar el peso de cada desenlace a lo largo de la temporada.

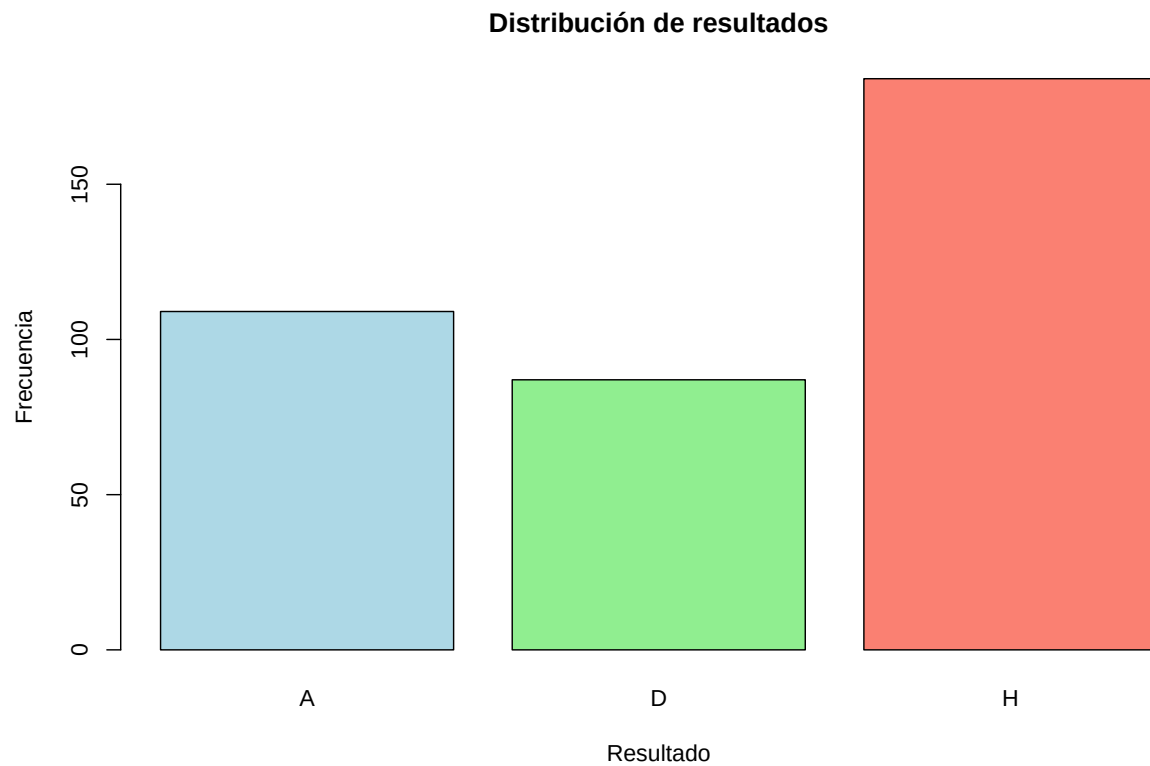
```
# Distribución de resultados
table(datos$FTR)
```

```
##
##      A      D      H
## 109   87   184
```

```
prop.table(table(datos$FTR))
```

```
##
##           A           D           H
## 0.2868421 0.2289474 0.4842105
```

```
# Visualización con el gráfico de barras
barplot(table(datos$FTR),
        col = c("lightblue", "lightgreen", "salmon"),
        main = "Distribución de resultados",
        xlab = "Resultado",
        ylab = "Frecuencia")
```



Los resultados muestran que la victoria local es el resultado más frecuente con un 48.4% de los partidos, seguida de la victoria visitante con un 28.7% y el empate con un 22.9%, lo que sugiere la existencia de una clara ventaja de jugar en casa.

3.3. Comparación entre equipo local y visitante

En esta subsección se analiza la diferencia en el rendimiento ofensivo entre equipos locales y visitantes, con el objetivo de identificar posibles patrones asociados al factor campo.

En lugar de repetir las variables ya analizadas anteriormente, se profundiza en indicadores clave como los tiros a puerta y la eficiencia, que permiten evaluar tanto la generación como la calidad de las ocasiones.

```
mean(datos$HST) # Media de tiros a puerta de local
```

```
## [1] 4.907895
```

```
mean(datos$AST) # Media de tiros a puerta de visitante
```

```
## [1] 3.894737
```

```
mean(datos$EfficiencyHome) # Media de eficiencia ofensiva de local
```

```
## [1] 0.3381971
```

```
mean(datos$EfficiencyAway) # Media de eficiencia ofensiva de visitante
```

```
## [1] 0.3124512
```

Los resultados muestran que los equipos locales generan un mayor número de tiros a puerta (4.91) en comparación con los visitantes (3.89), lo que confirma una mayor capacidad de generar ocasiones claras de gol gracias al factor campo; sin embargo, al analizar la eficiencia ofensiva, se observa que las diferencias son mínimas (33.8% local frente a 31.2% visitante), lo que sugiere que ambos presentan una capacidad similar para transformar sus oportunidades en goles, concluyendo así que la ventaja de jugar en casa está directamente relacionada con un mayor volumen de ataque y dominio del juego ofensivo más que con una precisión superior en la finalización de las jugadas.

3.4. Relación entre variables ofensivas y goles

En esta subsección se analiza la relación entre las variables ofensivas y el número de goles anotados, con el objetivo de comprender hasta qué punto la generación de ocasiones influye en el rendimiento ofensivo de los equipos.

En particular, se estudia la relación entre el número de tiros, los tiros a puerta y los goles anotados, tanto para el equipo local como para el visitante.

```
# Gráficos de dispersión (en rejilla 2x2)
par(mfrow = c(2, 2))
plot(datos$HS, datos$FTHG,
     main = "Relación entre tiros y goles (local)",
     xlab = "Tiros totales",
     ylab = "Goles",
     pch = 16,
     col = "blue")

# Gráfico de relación entre tiros a puerta y goles (local)
plot(datos$HST, datos$FTHG,
     main = "Relación entre tiros a puerta y goles (local)",
     xlab = "Tiros a puerta",
     ylab = "Goles",
     pch = 16,
     col = "darkblue")

# Correlaciones (equipo local)
cor(datos$HS, datos$FTHG)
```

```
## [1] 0.3346944
```

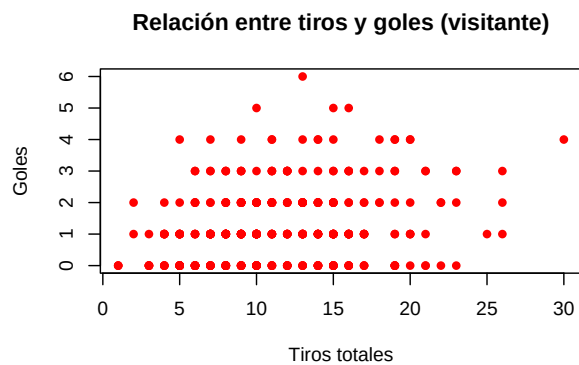
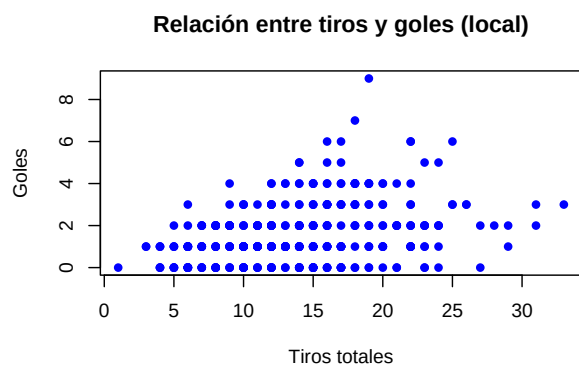
```
cor(datos$HST, datos$FTHG)
```

```
## [1] 0.5914285
```

```
# Gráfico de relación entre tiros y goles (equipo visitante)
plot(datos$AS, datos$FTAG,
     main = "Relación entre tiros y goles (visitante)",
     xlab = "Tiros totales",
```

```
ylab = "Goles",
pch = 16,
col = "red")
```

```
# Gráfico de relación entre tiros a puerta y goles (visitante)
plot(datos$AST, datos$FTAG,
      main = "Relación entre tiros a puerta y goles (visitante)",
      xlab = "Tiros a puerta",
      ylab = "Goles",
      pch = 16,
      col = "darkred")
```



```
# Correlaciones (equipo visitante)
cor(datos$AS, datos$FTAG)
```

```
## [1] 0.2523152
```

```
cor(datos$AST, datos$FTAG)
```

```
## [1] 0.5404367
```

```
par(mfrow = c(1, 1))
```

Los gráficos de dispersión (plots) de la sección corroboran visualmente que la relación entre los tiros a puerta y los goles es mucho más lineal y agrupada que la de los tiros totales, la cual presenta una gran dispersión de puntos que indica ataques ineficientes. Esta diferencia se refleja en las correlaciones calculadas, donde el equipo local pasa de un 0.33 en tiros totales a un 0.59 en tiros a puerta, mientras que el visitante sube de 0.25 a 0.54. Al observar los ejes de los gráficos, se confirma que los equipos locales no solo muestran una asociación más estrecha entre sus intentos y el marcador, sino que mantienen un volumen ofensivo superior, con una media de 1.63 goles frente al 1.22 de los visitantes. En definitiva, los plots demuestran que la precisión y la calidad de las ocasiones (tiros a puerta) son los factores realmente determinantes para explicar el éxito goleador en la Premier League, por encima del simple volumen de disparos realizados.

3.5. Correlaciones entre variables numéricas

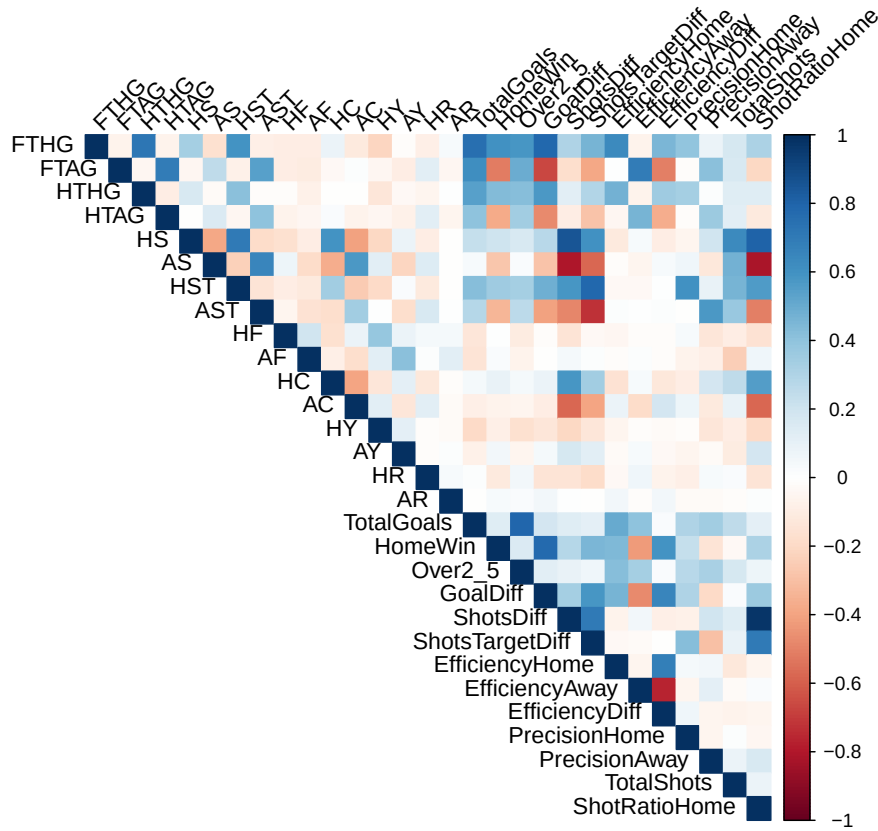
En esta subsección se analizan las correlaciones entre las variables numéricas del dataset, con el objetivo de identificar relaciones lineales entre diferentes aspectos del juego.

El estudio de las correlaciones permite comprender qué variables están más relacionadas entre sí, facilitando la interpretación del comportamiento del dataset y ayudando en la selección de variables para modelos posteriores.

```
# Selección de variables numéricas
vars_numericas <- datos[, sapply(datos, is.numeric)]

# Matriz de correlaciones
correlaciones <- cor(vars_numericas)

# Visualización gráfica de las correlaciones (corrplot ya cargado en 2.1)
corrplot(correlaciones,
          method = "color",
          type = "upper",
          tl.col = "black",
          tl.srt = 45)
```



El mapa de correlaciones permite visualizar de forma clara las relaciones entre las variables numéricas del dataset. En primer lugar, se observan fuertes correlaciones positivas entre variables directamente relacionadas, como los goles del equipo local y visitante con el número total de goles, lo cual es esperable debido a su propia definición. Asimismo, se aprecia una relación positiva entre el número de tiros y los tiros a puerta, lo que indica que los equipos que generan más ocasiones tienden también a producir un mayor número de disparos efectivos. Por otro lado, las variables relacionadas con la eficiencia y la precisión presentan correlaciones más moderadas, lo que sugiere que la calidad de las ocasiones no depende exclusivamente del volumen de tiros realizados. Además, se observan algunas correlaciones negativas entre variables de equipos opuestos, reflejando la naturaleza competitiva del juego, donde el dominio de un equipo suele implicar menor rendimiento del rival. En conjunto, el análisis confirma que las variables ofensivas están interrelacionadas, siendo especialmente relevantes aquellas que miden la calidad de las ocasiones generadas.

4. ¿Existe realmente la ventaja de jugar en casa?

En esta sección se aplican técnicas de inferencia estadística con el objetivo de validar de forma rigurosa si la ventaja de jugar en casa es estadísticamente significativa o si podría deberse al azar.

Para ello, se emplean intervalos de confianza y contrastes de hipótesis sobre variables clave del estudio, como el número de goles, los tiros a puerta y la relación entre distintos factores del juego y el resultado de los partidos.

4.1. Intervalo de confianza para la media de goles locales

En esta subsección se estima la media de goles anotados por los equipos locales mediante la construcción de un intervalo de confianza. El objetivo es obtener un rango en el que se encuentre la media poblacional con

un determinado nivel de confianza, en este caso del 95%.

```
# Intervalo de confianza
ic_goles_local <- t.test(datos$FTHG, conf.level = 0.95)
ic_goles_local
```

```
##
## One Sample t-test
##
## data: datos$FTHG
## t = 22.435, df = 379, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  1.490986 1.777435
## sample estimates:
## mean of x
##  1.634211
```

```
# Media
mean(datos$FTHG)
```

```
## [1] 1.634211
```

```
# Desviación típica
sd(datos$FTHG)
```

```
## [1] 1.419944
```

```
# Tamaño muestra
length(datos$FTHG)
```

```
## [1] 380
```

El intervalo de confianza obtenido permite estimar la media real de goles anotados por los equipos locales en la competición.

Con un nivel de confianza del 95%, se puede afirmar que la media poblacional se encuentra dentro del intervalo calculado [1.490986, 1.777435].

Dado que la media muestral es aproximadamente 1.63 goles por partido, los resultados reflejan una capacidad ofensiva destacada de los equipos locales.

Además, el intervalo obtenido presenta una amplitud moderada, lo que indica que la estimación es relativamente precisa a pesar de la variabilidad existente.

4.2. Contraste de medias: goles local vs visitante

En esta subsección se analiza si existen diferencias significativas entre el número de goles anotados por los equipos locales y visitantes. Aunque en el análisis exploratorio ya se observaba que los equipos locales tienden a marcar más goles, es necesario validar si esta diferencia es estadísticamente significativa o si podría deberse al azar. Se plantea el siguiente contraste de hipótesis:

- H0: La media de goles de los equipos locales es igual a la de los visitantes.
- H1: La media de goles de los equipos locales es mayor que la de los visitantes.

```
# Test t de dos muestras (unilateral superior)
test_goles <- t.test(datos$FTHG, datos$FTAG, alternative = "greater")
test_goles
```

```
##
## Welch Two Sample t-test
##
## data: datos$FTHG and datos$FTAG
## t = 4.3848, df = 734.18, p-value = 6.654e-06
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  0.2596174      Inf
## sample estimates:
## mean of x mean of y
##  1.634211  1.218421
```

```
# Cálculo de medias para verificar
mean(datos$FTHG) # Media goles local
```

```
## [1] 1.634211
```

```
mean(datos$FTAG) # Media goles visitante
```

```
## [1] 1.218421
```

Dado que el p-valor (6.654e-06) es significativamente inferior al nivel de error permitido (0.05), existe evidencia estadística suficiente para rechazar la hipótesis nula.

Esto confirma que la diferencia entre la media de goles locales (1.63) y visitantes (1.22) no se debe al azar, validando de forma científica la existencia de una ventaja de jugar en casa en esta competición. En promedio, los equipos locales anotan 0.42 goles más por partido que los visitantes (diferencia de 0.4158).

En esta subsección se analiza si existen diferencias significativas entre el número de tiros a puerta realizados por los equipos locales y visitantes. Al igual que en el caso de los goles, los equipos locales generan más ocasiones, pero es necesario validar si esta diferencia es estadísticamente significativa. Hipótesis:

- H0: la media de tiros a puerta es igual para ambos
- H1: la media de tiros a puerta del equipo local es mayor que la del visitante

```
# Test t para comparar medias de tiros a puerta (unilateral superior)
test_tp <- t.test(datos$HST, datos$AST,
                  alternative = "greater")
test_tp
```

```
##
## Welch Two Sample t-test
##
## data: datos$HST and datos$AST
```

```
## t = 5.9009, df = 748.67, p-value = 2.737e-09
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  0.7303957      Inf
## sample estimates:
## mean of x mean of y
##  4.907895  3.894737
```

```
# Medias de tiros a puerta
mean(datos$HST) # local
```

```
## [1] 4.907895
```

```
mean(datos$AST) # visitante
```

```
## [1] 3.894737
```

Al realizar el contraste de medias para los tiros a puerta, se ha obtenido un p-valor de 2.737e-09, el cual es muy inferior al nivel de significancia de 0.05, lo que obliga a rechazar la hipótesis nula y confirmar que existe una diferencia estadísticamente significativa entre ambos grupos. Los resultados muestran que los equipos locales generan un mayor volumen ofensivo con una media de 4.91 tiros a puerta por partido frente a los 3.89 de los visitantes, con un intervalo de confianza que indica que la ventaja local es de al menos 0.73 disparos. Esta evidencia estadística confirma un mayor dominio de los equipos de casa en la generación de ocasiones claras, reforzando la conclusión de que el factor campo influye de manera determinante en el comportamiento ofensivo durante la competición.

4.4. Asociación entre tarjetas rojas y resultado

En esta subsección se analiza si existe relación entre la presencia de tarjetas rojas y el resultado final del partido. A diferencia de los apartados anteriores, en este caso se trabaja con variables categóricas, por lo que se emplea el test Chi-cuadrado. El test de Chi-cuadrado permite analizar si existe dependencia entre la presencia de tarjetas rojas y el resultado del partido. Si el p-valor es inferior a 0.05, se rechaza la hipótesis nula, concluyendo que existe relación entre ambas variables. En ese caso, se puede afirmar que las expulsiones influyen en el resultado, ya que alteran el equilibrio del partido. Si el p-valor es superior a 0.05, no se puede afirmar que exista una relación significativa entre las variables analizadas.

```
# Crear variable: presencia de expulsión
datos$RedCard <- ifelse(datos$HR + datos$AR > 0, 1, 0)
datos$RedCard <- as.factor(datos$RedCard)
```

```
# Tabla de contingencia
tabla_red <- table(datos$FTR, datos$RedCard)
tabla_red
```

```
##
##      0      1
## A 100      9
## D  80      7
## H 174     10
```

```
# Proporciones
prop.table(tabla_red, margin = 2)
```

```
##
##           0           1
##  A 0.2824859 0.3461538
##  D 0.2259887 0.2692308
##  H 0.4915254 0.3846154
```

```
# Test Chi-cuadrado
test_chi <- chisq.test(tabla_red)
test_chi
```

```
##
## Pearson's Chi-squared test
##
## data:  tabla_red
## X-squared = 1.1118, df = 2, p-value = 0.5735
```

El test de Chi-cuadrado permite analizar si existe dependencia entre la presencia de tarjetas rojas y el resultado del partido. Si el p-valor es superior a 0.05, no se puede afirmar que exista una relación significativa entre las variables analizadas. Dado que el p-valor obtenido es de 0.5735, el cual es superior al nivel de significancia de 0.05, no se puede rechazar la hipótesis nula, concluyendo que no se dispone de evidencia suficiente para afirmar que exista una relación estadísticamente significativa entre la presencia de expulsiones y el resultado final del partido en esta muestra.

5. ¿Cuánto de bueno es cada equipo realmente?

En esta sección se aplica el modelo Dixon-Coles, un modelo estadístico de referencia en la analítica de fútbol, con el objetivo de estimar la fuerza ofensiva y la debilidad defensiva de cada equipo de forma independiente al resultado final de la temporada.

A diferencia de la clasificación oficial, que mezcla calidad y suerte, este modelo permite separar ambos factores, asignando a cada equipo dos parámetros latentes: su capacidad de ataque y su solidez defensiva. Además, se incorpora un parámetro de ventaja local que captura el efecto del factor campo de forma explícita.

La estimación de los parámetros se realiza por máxima verosimilitud, asumiendo que los goles de cada equipo siguen distribuciones de Poisson independientes cuyas medias dependen de la fuerza de ataque del equipo que marca y de la debilidad defensiva del equipo que encaja.

5.1. Modelo Dixon-Coles: estimación por máxima verosimilitud

En esta subsección se construye y estima el modelo Dixon-Coles. La idea central es modelizar los goles de cada equipo como variables aleatorias de Poisson cuya media depende de tres factores: la fuerza de ataque del equipo que marca, la debilidad defensiva del equipo que encaja, y la ventaja de jugar en casa. Los parámetros del modelo se estiman por máxima verosimilitud mediante optimización numérica.

```
# Equipos de la temporada
equipos <- levels(datos$HomeTeam)
n_equipos <- length(equipos)
```

```

# Función de log-verosimilitud negativa del modelo Dixon-Coles
log_verosimilitud <- function(params) {

  # Extraemos los parámetros: ataque, defensa y ventaja local
  ataque <- params[1:n equipos]
  defensa <- params[(n equipos + 1):(2 * n equipos)]
  ventaja <- params[2 * n equipos + 1]

  # Asignamos nombres para identificar cada equipo
  names(ataque) <- equipos
  names(defensa) <- equipos

  log_ver <- 0

  for (i in 1:nrow(datos)) {

    home <- as.character(datos$HomeTeam[i])
    away <- as.character(datos$AwayTeam[i])

    # Media de goles esperada para cada equipo
    lambda_home <- exp(ventaja + ataque[home] - defensa[away])
    lambda_away <- exp(ataque[away] - defensa[home])

    # Log-verosimilitud de Poisson para los goles observados
    log_ver <- log_ver +
      dpois(datos$FTHG[i], lambda_home, log = TRUE) +
      dpois(datos$FTAG[i], lambda_away, log = TRUE)
  }

  return(-log_ver) # Devolvemos el negativo para minimizar
}

# Valores iniciales de los parámetros
params_iniciales <- c(
  rep(0, n equipos), # ataque
  rep(0, n equipos), # defensa
  0.1                # ventaja local
)

# Estimación por máxima verosimilitud mediante optim()
resultado <- optim(
  par      = params_iniciales,
  fn       = log_verosimilitud,
  method   = "BFGS",
  control  = list(maxit = 10000)
)

# Extraemos los parámetros estimados
ataque_est <- resultado$par[1:n equipos]
defensa_est <- resultado$par[(n equipos + 1):(2 * n equipos)]
ventaja_est <- resultado$par[2 * n equipos + 1]

names(ataque_est) <- equipos

```

```
names(defensa_est) <- equipos
```

```
# Ventaja local estimada  
round(ventaja_est, 4)
```

```
## [1] 0.2937
```

La ventaja local estimada es de 0.2937, lo que implica que jugar en casa multiplica por $\exp(0.2937) = 1.34$ los goles esperados de media. Es decir, un equipo marca aproximadamente un 34% más de goles jugando en casa que fuera, independientemente de su calidad. Este resultado es consistente con los contrastes de hipótesis de la sección 4, donde ya se confirmó estadísticamente la existencia de ventaja local.

5.2. Ranking de fuerza de ataque y debilidad defensiva

En esta subsección se construye una tabla con los parámetros estimados para cada equipo. Un valor de ataque alto indica mayor capacidad ofensiva. Un valor de defensa bajo indica mayor solidez defensiva, ya que el parámetro de defensa representa la debilidad: cuanto menor, más difícil es marcarle al equipo.

```
# Tabla de parámetros por equipo  
tabla_dc <- data.frame(  
  Equipo = equipos,  
  Ataque = round(ataque_est, 4),  
  Defensa = round(defensa_est, 4)  
)  
  
# Ranking por fuerza de ataque de mayor a menor  
tabla_dc_ataque <- tabla_dc[order(-tabla_dc$Ataque), ]  
cat("\nRanking por fuerza de ataque:\n")
```

```
##  
## Ranking por fuerza de ataque:
```

```
print(tabla_dc_ataque)
```

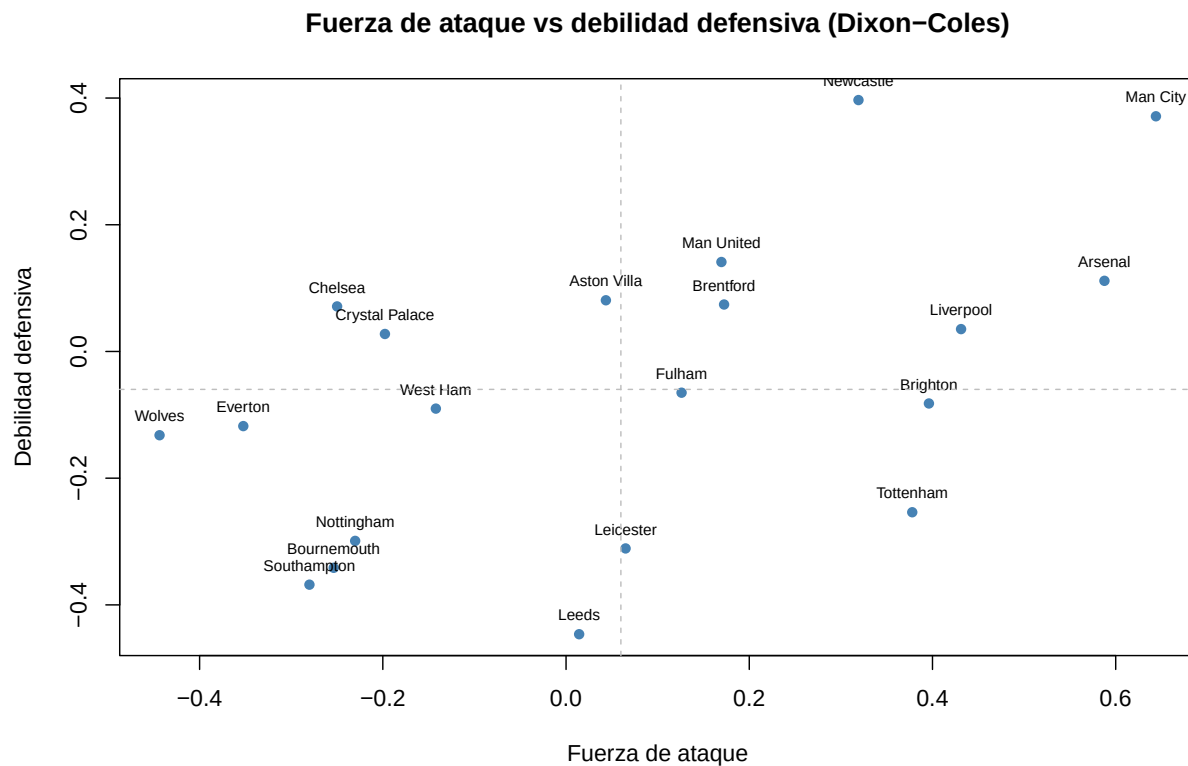
```
##           Equipo  Ataque  Defensa  
## Man City      Man City  0.6439  0.3712  
## Arsenal       Arsenal  0.5877  0.1115  
## Liverpool     Liverpool 0.4312  0.0353  
## Brighton      Brighton 0.3961 -0.0821  
## Tottenham     Tottenham 0.3779 -0.2538  
## Newcastle     Newcastle 0.3192  0.3968  
## Brentford     Brentford 0.1725  0.0741  
## Man United    Man United 0.1696  0.1412  
## Fulham        Fulham   0.1261 -0.0651  
## Leicester     Leicester 0.0650 -0.3109  
## Aston Villa   Aston Villa 0.0435  0.0808  
## Leeds         Leeds    0.0143 -0.4463  
## West Ham      West Ham -0.1421 -0.0902  
## Crystal Palace Crystal Palace -0.1977  0.0276  
## Nottingham    Nottingham -0.2302 -0.2990
```

```
## Chelsea          Chelsea -0.2499  0.0711
## Bournemouth      Bournemouth -0.2536 -0.3415
## Southampton      Southampton -0.2801 -0.3681
## Everton          Everton -0.3523 -0.1178
## Wolves           Wolves -0.4437 -0.1322
```

```
# Ranking por solidez defensiva de menor a mayor
tabla_dc_defensa <- tabla_dc[order(tabla_dc$Defensa), ]
print(tabla_dc_defensa)
```

```
##           Equipo  Ataque Defensa
## Leeds           Leeds  0.0143 -0.4463
## Southampton     Southampton -0.2801 -0.3681
## Bournemouth     Bournemouth -0.2536 -0.3415
## Leicester       Leicester  0.0650 -0.3109
## Nottingham      Nottingham -0.2302 -0.2990
## Tottenham       Tottenham  0.3779 -0.2538
## Wolves          Wolves -0.4437 -0.1322
## Everton         Everton -0.3523 -0.1178
## West Ham        West Ham -0.1421 -0.0902
## Brighton        Brighton  0.3961 -0.0821
## Fulham          Fulham  0.1261 -0.0651
## Crystal Palace  Crystal Palace -0.1977  0.0276
## Liverpool       Liverpool  0.4312  0.0353
## Chelsea         Chelsea -0.2499  0.0711
## Brentford       Brentford  0.1725  0.0741
## Aston Villa     Aston Villa  0.0435  0.0808
## Arsenal         Arsenal  0.5877  0.1115
## Man United      Man United  0.1696  0.1412
## Man City        Man City  0.6439  0.3712
## Newcastle       Newcastle  0.3192  0.3968
```

```
# Visualización conjunta: ataque vs defensa
plot(tabla_dc$Ataque, tabla_dc$Defensa,
     main = "Fuerza de ataque vs debilidad defensiva (Dixon-Coles)",
     xlab = "Fuerza de ataque",
     ylab = "Debilidad defensiva",
     pch = 16,
     col = "steelblue")
text(tabla_dc$Ataque, tabla_dc$Defensa,
     labels = tabla_dc$Equipo,
     cex = 0.7,
     pos = 3)
abline(h = mean(tabla_dc$Defensa), lty = 2, col = "gray")
abline(v = mean(tabla_dc$Ataque), lty = 2, col = "gray")
```



El gráfico revela cuatro perfiles claramente diferenciados. Man City y Arsenal destacan como los equipos más completos, combinando alto ataque con baja debilidad defensiva, lo que los sitúa como los candidatos reales al título. Brighton y Liverpool también presentan una buena combinación de ambos parámetros. Newcastle llama la atención por su alto ataque pero también alta debilidad defensiva, perfil de equipo ofensivo pero frágil atrás. En el extremo opuesto, Southampton, Bournemouth y Leeds combinan bajo ataque con alta debilidad defensiva, lo que explica sus problemas para mantenerse en la categoría. Wolves y Everton presentan bajo ataque pero buena solidez defensiva, perfil típico de equipos que compiten bien en la tabla pero sin capacidad real de amenazar los puestos europeos.

5.3. Comparación con la clasificación real

En esta subsección se comparan los parámetros estimados por el modelo Dixon-Coles con la clasificación real de la temporada 2022-2023. El objetivo es evaluar en qué medida la calidad estimada por el modelo coincide con el rendimiento real, y detectar equipos que rindieron por encima o por debajo de su nivel esperado.

```
# Cálculo de los puntos reales de cada equipo
puntos_real <- data.frame(Equipo = equipos, Puntos = 0)

for (i in 1:nrow(datos)) {
  home <- as.character(datos$HomeTeam[i])
  away <- as.character(datos$AwayTeam[i])
  res <- as.character(datos$FTR[i])

  if (res == "H") {
    puntos_real$Puntos[puntos_real$Equipo == home] <-
      puntos_real$Puntos[puntos_real$Equipo == home] + 3
  }
}
```

```

} else if (res == "A") {
  puntos_real$Puntos[puntos_real$Equipo == away] <-
    puntos_real$Puntos[puntos_real$Equipo == away] + 3
} else {
  puntos_real$Puntos[puntos_real$Equipo == home] <-
    puntos_real$Puntos[puntos_real$Equipo == home] + 1
  puntos_real$Puntos[puntos_real$Equipo == away] <-
    puntos_real$Puntos[puntos_real$Equipo == away] + 1
}
}

# Añadimos posición en la clasificación real
puntos_real <- puntos_real[order(-puntos_real$Puntos), ]
puntos_real$Posicion_Real <- 1:nrow(puntos_real)

# Unimos con los parámetros Dixon-Coles
comparacion <- merge(puntos_real, tabla_dc, by = "Equipo")

# Ranking Dixon-Coles por ataque
comparacion <- comparacion[order(-comparacion$Ataque), ]
comparacion$Posicion_DC <- 1:nrow(comparacion)

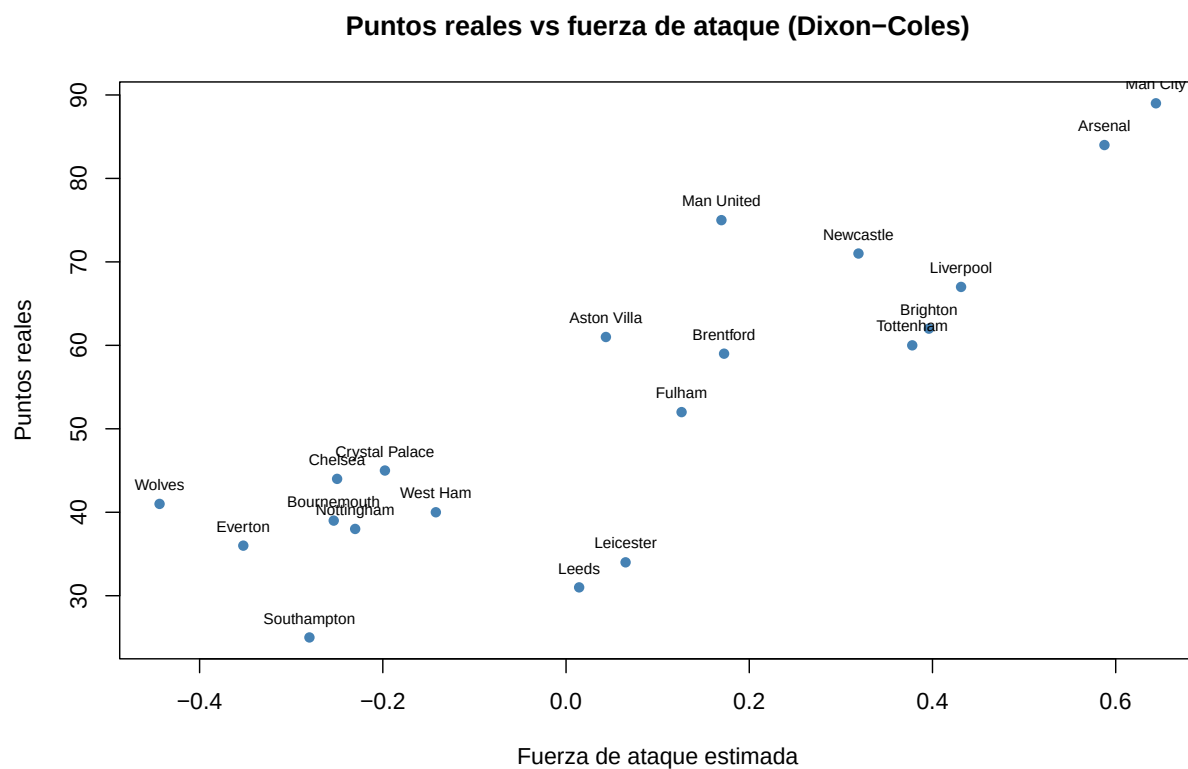
# Diferencia entre posición real y posición Dixon-Coles
comparacion$Diferencia <- comparacion$Posicion_Real - comparacion$Posicion_DC

# Tabla ordenada por clasificación real
comparacion <- comparacion[order(comparacion$Posicion_Real), ]
print(comparacion[, c("Equipo", "Puntos", "Posicion_Real",
  "Posicion_DC", "Diferencia")])

```

##	Equipo	Puntos	Posicion_Real	Posicion_DC	Diferencia
## 13	Man City	89	1	1	0
## 1	Arsenal	84	2	2	0
## 14	Man United	75	3	8	-5
## 15	Newcastle	71	4	6	-2
## 12	Liverpool	67	5	3	2
## 5	Brighton	62	6	4	2
## 2	Aston Villa	61	7	11	-4
## 18	Tottenham	60	8	5	3
## 4	Brentford	59	9	7	2
## 9	Fulham	52	10	9	1
## 7	Crystal Palace	45	11	14	-3
## 6	Chelsea	44	12	16	-4
## 20	Wolves	41	13	20	-7
## 19	West Ham	40	14	13	1
## 3	Bournemouth	39	15	17	-2
## 16	Nottingham	38	16	15	1
## 8	Everton	36	17	19	-2
## 11	Leicester	34	18	10	8
## 10	Leeds	31	19	12	7
## 17	Southampton	25	20	18	2

```
# Visualización: puntos reales vs fuerza de ataque estimada
plot(comparacion$Ataque, comparacion$Puntos,
     main = "Puntos reales vs fuerza de ataque (Dixon-Coles)",
     xlab = "Fuerza de ataque estimada",
     ylab = "Puntos reales",
     pch = 16,
     col = "steelblue")
text(comparacion$Ataque, comparacion$Puntos,
     labels = comparacion$Equipo,
     cex = 0.7,
     pos = 3)
```



Los resultados muestran que Man City y Arsenal coinciden perfectamente entre su posición real y la estimada por Dixon-Coles, confirmando que su rendimiento estuvo completamente alineado con su calidad real. Los casos más llamativos son Leicester y Leeds, que terminaron mucho más abajo en la clasificación real de lo que su calidad estimada sugería, lo que apunta a mala suerte o ineficiencia en momentos clave. En sentido contrario, Wolves y Aston Villa rindieron por encima de su calidad estimada. Man United también aparece sobrerendiendo: el modelo estima que su nivel ofensivo no justifica un tercer puesto. Estas diferencias motivan directamente la sección siguiente, donde la simulación Monte Carlo cuantificará el papel exacto de la suerte en la clasificación final.

6. ¿Quién habría ganado la liga en otras circunstancias?

En esta sección se simula la temporada 2022-2023 de la Premier League 10.000 veces utilizando los parámetros estimados por el modelo Dixon-Coles. El objetivo es cuantificar el papel de la suerte en la clasificación final, obteniendo distribuciones de probabilidad para cada equipo de ser campeón, terminar en el top 4 o descender.

6.1. Simulación Monte Carlo de la temporada (10.000 iteraciones)

En esta subsección se implementa la simulación Monte Carlo. Para cada iteración se simulan los 380 partidos de la temporada generando marcadores aleatorios mediante distribuciones de Poisson con los parámetros estimados por Dixon-Coles. A partir de cada simulación se obtiene una clasificación final completa.

```
set.seed(42) # Fijamos semilla para reproducibilidad
n_sim <- 10000 # Número de simulaciones

# Matriz para almacenar las posiciones finales de cada equipo en cada simulación
posiciones <- matrix(0, nrow = n_sim, ncol = n_equipos)
colnames(posiciones) <- equipos

for (s in 1:n_sim) {

  # Mensaje de progreso cada 1000 simulaciones
  if (s %% 1000 == 0) cat("Simulación", s, "de", n_sim, "\n")

  # Inicializamos los puntos de cada equipo en 0
  puntos_sim <- setNames(rep(0, n_equipos), equipos)

  for (i in 1:nrow(datos)) {

    home <- as.character(datos$HomeTeam[i])
    away <- as.character(datos$AwayTeam[i])

    # Medias de goles esperadas según Dixon-Coles
    lambda_home <- exp(ventaja_est + ataque_est[home] - defensa_est[away])
    lambda_away <- exp(ataque_est[away] - defensa_est[home])

    # Simulación de goles mediante distribución de Poisson
    goles_home <- rpois(1, lambda_home)
    goles_away <- rpois(1, lambda_away)

    # Asignación de puntos según resultado simulado
    if (goles_home > goles_away) {
      puntos_sim[home] <- puntos_sim[home] + 3
    } else if (goles_away > goles_home) {
      puntos_sim[away] <- puntos_sim[away] + 3
    } else {
      puntos_sim[home] <- puntos_sim[home] + 1
      puntos_sim[away] <- puntos_sim[away] + 1
    }
  }
}

# Guardamos la posición final de cada equipo en esta simulación
```

```
posiciones[s, ] <- rank(-puntos_sim, ties.method = "random")
}
```

```
## Simulación 1000 de 10000
## Simulación 2000 de 10000
## Simulación 3000 de 10000
## Simulación 4000 de 10000
## Simulación 5000 de 10000
## Simulación 6000 de 10000
## Simulación 7000 de 10000
## Simulación 8000 de 10000
## Simulación 9000 de 10000
## Simulación 10000 de 10000
```

(El archivo pesado .rds ya no se exporta: el repo solo necesita la tabla 20x4 de probabilidades, que se escribe en 6.2 como montecarlo_probs.csv.)

La simulación genera 10.000 temporadas alternativas partiendo de la misma calidad estimada para cada equipo pero con resultados aleatorios en cada partido. Esto permite obtener distribuciones de probabilidad completas para cada posición de la clasificación final.

6.2. Probabilidades de campeón, top 4 y descenso

En esta subsección se calculan las probabilidades de cada equipo de ser campeón, terminar en el top 4 (clasificación europea) y descender (posiciones 18, 19 y 20) a partir de las 10.000 simulaciones.

(posiciones ya está en memoria desde 6.1 dentro de la misma sesión)

```
# Probabilidad de ser campeón (posición 1)
prob_campeon <- colMeans(posiciones == 1) * 100

# Probabilidad de top 4
prob_top4 <- colMeans(posiciones <= 4) * 100

# Probabilidad de descenso (posiciones 18, 19, 20)
prob_descenso <- colMeans(posiciones >= 18) * 100

# Tabla de resultados
tabla_mc <- data.frame(
  Equipo = equipos,
  Campeon = round(prob_campeon, 1),
  Top4 = round(prob_top4, 1),
  Descenso = round(prob_descenso, 1)
)

# Ordenamos por probabilidad de campeón
tabla_mc <- tabla_mc[order(-tabla_mc$Campeon), ]

# PUENTE R -> Python: exportar SOLO la tabla resumen 20x4 de probabilidades
write.csv(tabla_mc, "montecarlo_probs.csv", row.names = FALSE)
cat("\nProbabilidades Monte Carlo (%):\n")
```

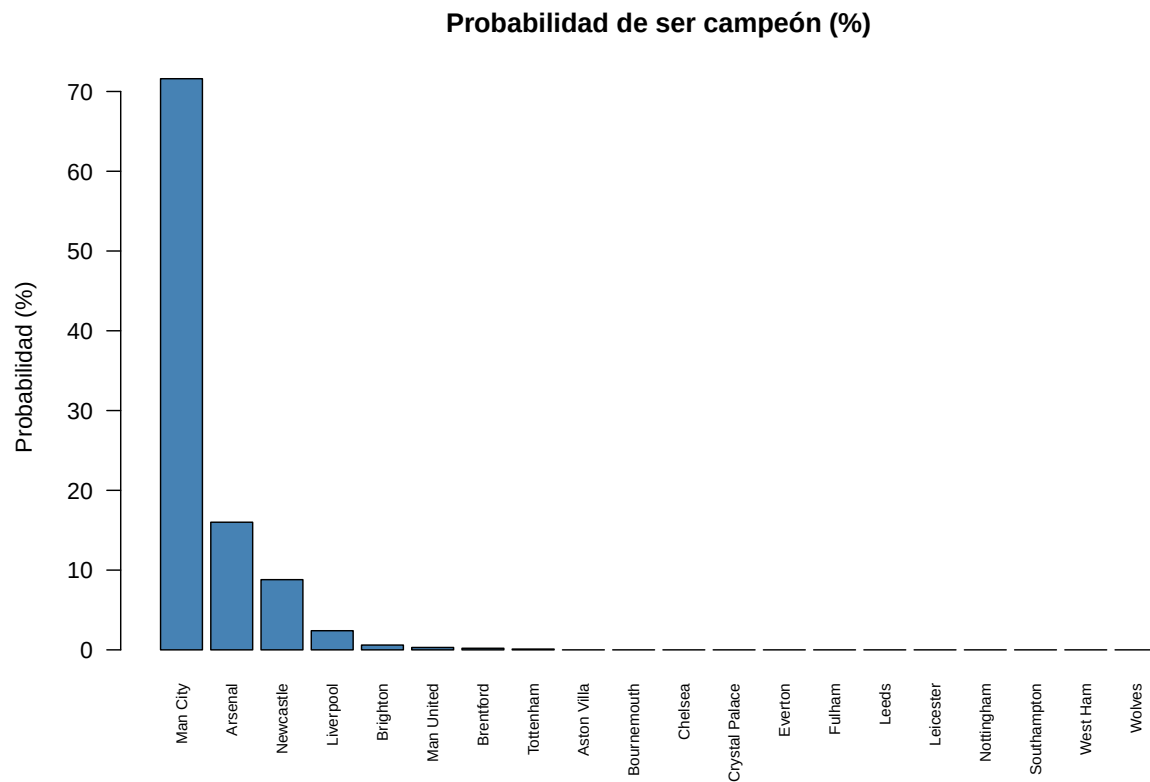
```
##
## Probabilidades Monte Carlo (%):
```

```
print(tabla_mc)
```

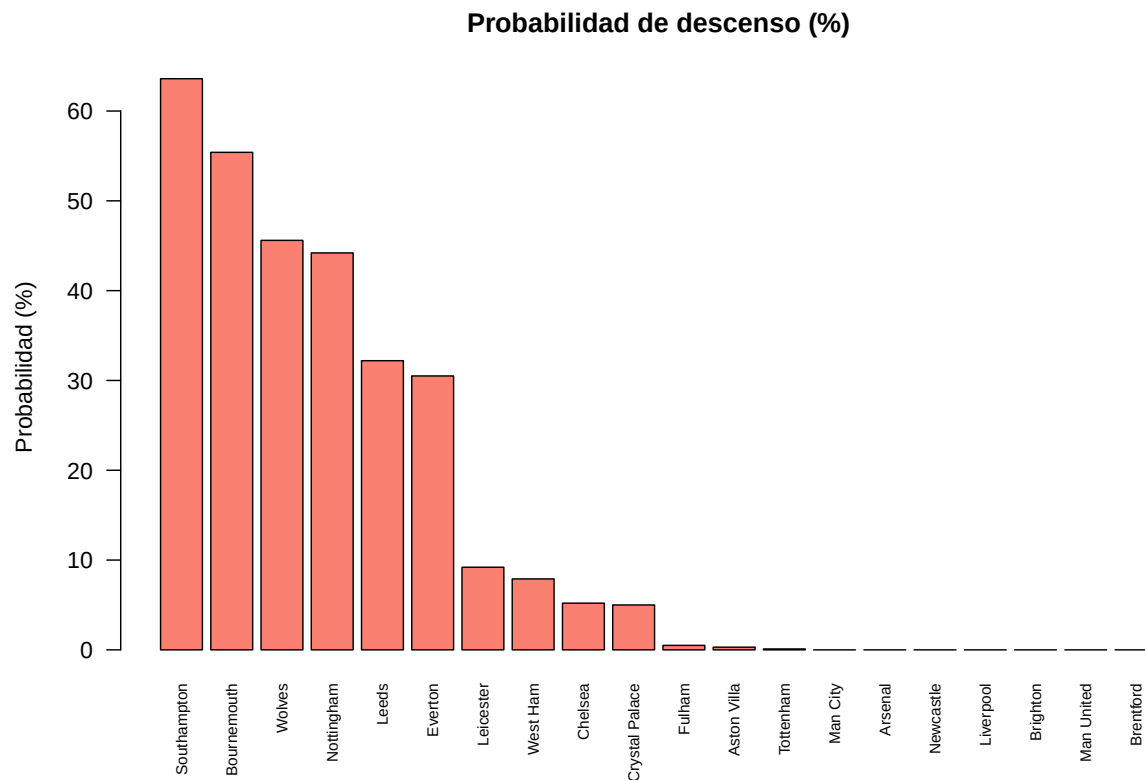
	Equipo	Campeon	Top4	Descenso	
##	Man City	Man City	71.6	99.3	0.0
##	Arsenal	Arsenal	16.0	90.1	0.0
##	Newcastle	Newcastle	8.8	80.9	0.0
##	Liverpool	Liverpool	2.4	53.7	0.0
##	Brighton	Brighton	0.6	26.7	0.0
##	Man United	Man United	0.3	19.8	0.0
##	Brentford	Brentford	0.2	13.1	0.0
##	Tottenham	Tottenham	0.1	7.2	0.1
##	Aston Villa	Aston Villa	0.0	5.1	0.3
##	Bournemouth	Bournemouth	0.0	0.0	55.4
##	Chelsea	Chelsea	0.0	0.2	5.2
##	Crystal Palace	Crystal Palace	0.0	0.2	5.0
##	Everton	Everton	0.0	0.0	30.5
##	Fulham	Fulham	0.0	3.5	0.5
##	Leeds	Leeds	0.0	0.0	32.2
##	Leicester	Leicester	0.0	0.1	9.2
##	Nottingham	Nottingham	0.0	0.0	44.2
##	Southampton	Southampton	0.0	0.0	63.6
##	West Ham	West Ham	0.0	0.1	7.9
##	Wolves	Wolves	0.0	0.0	45.6

```
# Visualización: probabilidad de campeón
```

```
barplot(tabla_mc$Campeon,  
        names.arg = tabla_mc$Equipo,  
        main = "Probabilidad de ser campeón (%)",  
        ylab = "Probabilidad (%)",  
        col = "steelblue",  
        las = 2,  
        cex.names = 0.7)
```



```
# Visualización: probabilidad de descenso
tabla_desc <- tabla_mc[order(-tabla_mc$Descenso), ]
barplot(tabla_desc$Descenso,
        names.arg = tabla_desc$Equipo,
        main = "Probabilidad de descenso (%)",
        ylab = "Probabilidad (%)",
        col = "salmon",
        las = 2,
        cex.names = 0.7)
```



Los resultados de la simulación revelan una jerarquía muy clara en la Premier League 2022-2023. Man City fue campeón en el 71.6% de las simulaciones, confirmando que su título no fue casualidad sino una superioridad real y contundente sobre el resto. Arsenal aparece como el único rival real con un 16.0% de probabilidad de título y un 90.1% de terminar en el top 4, lo que confirma que la lucha por el campeonato fue genuinamente disputada entre estos dos equipos. Newcastle sorprende con un 8.8% de probabilidad de título, siendo el tercer equipo con más opciones reales según el modelo. En el extremo inferior, Southampton presenta la mayor probabilidad de descenso con un 63.6%, seguida de Bournemouth con un 55.4% y Wolves con un 45.6%.

6.3. Separando calidad de fortuna

En esta subsección se compara la clasificación real con las probabilidades estimadas por Monte Carlo para identificar equipos que tuvieron suerte positiva o negativa a lo largo de la temporada.

```
# Unimos la tabla Monte Carlo con la clasificación real
tabla_final <- merge(tabla_mc, puntos_real[, c("Equipo", "Puntos", "Posicion_Real")],
                     by = "Equipo")
tabla_final <- tabla_final[order(tabla_final$Posicion_Real), ]

cat("\nClasificación real vs probabilidades Monte Carlo:\n")

##
## Clasificación real vs probabilidades Monte Carlo:
```

```
print(tabla_final[, c("Equipo", "Puntos", "Posicion_Real", "Campeon", "Top4", "Descenso")])
```

##	Equipo	Puntos	Posicion_Real	Campeon	Top4	Descenso
## 13	Man City	89	1	71.6	99.3	0.0
## 1	Arsenal	84	2	16.0	90.1	0.0
## 14	Man United	75	3	0.3	19.8	0.0
## 15	Newcastle	71	4	8.8	80.9	0.0
## 12	Liverpool	67	5	2.4	53.7	0.0
## 5	Brighton	62	6	0.6	26.7	0.0
## 2	Aston Villa	61	7	0.0	5.1	0.3
## 18	Tottenham	60	8	0.1	7.2	0.1
## 4	Brentford	59	9	0.2	13.1	0.0
## 9	Fulham	52	10	0.0	3.5	0.5
## 7	Crystal Palace	45	11	0.0	0.2	5.0
## 6	Chelsea	44	12	0.0	0.2	5.2
## 20	Wolves	41	13	0.0	0.0	45.6
## 19	West Ham	40	14	0.0	0.1	7.9
## 3	Bournemouth	39	15	0.0	0.0	55.4
## 16	Nottingham	38	16	0.0	0.0	44.2
## 8	Everton	36	17	0.0	0.0	30.5
## 11	Leicester	34	18	0.0	0.1	9.2
## 10	Leeds	31	19	0.0	0.0	32.2
## 17	Southampton	25	20	0.0	0.0	63.6

La comparación entre la clasificación real y las probabilidades Monte Carlo revela casos muy llamativos. Man United terminó tercero en la clasificación real pero el modelo le asignaba solo un 0.3% de probabilidad de título y un 19.8% de top 4, lo que sugiere que su rendimiento estuvo muy por encima de su calidad real estimada. Wolves es el caso más extremo: terminó en posición 13 de la clasificación real pero el modelo le asignaba un 45.6% de probabilidad de descenso, lo que indica que salvaron la categoría con una suerte considerable. En sentido contrario, Leicester terminó en el puesto 18 con un descenso real pero el modelo solo le asignaba un 9.2% de probabilidad de descenso, lo que apunta a que su bajada estuvo influenciada por mala suerte o rendimiento muy por debajo de su nivel real. En conjunto, la simulación demuestra que aunque la calidad explica la mayor parte de la clasificación final, la suerte jugó un papel determinante en varios puestos de la tabla, especialmente en la zona media y en la lucha por la permanencia.

7. ¿Se puede predecir el resultado de un partido?

En esta sección se aplican diferentes técnicas de clasificación supervisada con el objetivo de predecir el resultado de un partido de fútbol. Se comparan cuatro enfoques distintos: regresión de Poisson para predecir goles, regresión logística binomial para predecir victoria local, análisis discriminante lineal (LDA) y clasificación mediante KNN. Los modelos se evalúan y comparan en la subsección final mediante matrices de confusión y validación cruzada.

7.1. Preparación y división de datos

En esta subsección se seleccionan las variables predictoras más relevantes y se dividen los datos en conjunto de entrenamiento (80%) y conjunto de prueba (20%), garantizando una evaluación objetiva del rendimiento de los modelos.

```

# Selección de variables predictoras
variables <- c("FTHG", "FTAG", "HS", "AS", "HST", "AST",
              "HC", "AC", "HF", "AF", "HY", "AY",
              "TotalGoals", "ShotsDiff", "ShotsTargetDiff",
              "EfficiencyHome", "EfficiencyAway",
              "PrecisionHome", "PrecisionAway")

# Fijamos semilla para reproducibilidad
set.seed(123)

# Índices de entrenamiento (80%)
idx_train <- sample(1:nrow(datos), size = 0.8 * nrow(datos))

# PUENTE R -> Python: exportar los índices para que Python use el MISMO split
write.csv(data.frame(idx = idx_train), "idx_train.csv", row.names = FALSE)

# Conjuntos de entrenamiento y prueba
train <- datos[idx_train, ]
test  <- datos[-idx_train, ]

nrow(train)

```

```
## [1] 304
```

```
nrow(test)
```

```
## [1] 76
```

La división de los datos en entrenamiento y prueba es fundamental para evaluar el rendimiento real de los modelos sobre datos no vistos durante el entrenamiento. El conjunto de entrenamiento con 304 partidos se utiliza para ajustar los modelos, mientras que el conjunto de prueba con 76 partidos se reserva exclusivamente para evaluar su capacidad predictiva. Los modelos se evalúan con métricas distintas según su naturaleza: la regresión de Poisson se evalúa con RMSE y MAE al predecir una variable continua (goles), mientras que la regresión logística, LDA y KNN se evalúan con accuracy y matriz de confusión al predecir una variable categórica (victoria o no).

7.2. Regresión de Poisson: predicción de goles

En esta subsección se ajusta un modelo de regresión de Poisson para predecir el número de goles anotados por el equipo local en función de las estadísticas del partido. La regresión de Poisson es la elección natural para modelizar variables de conteo como los goles, ya que asume que la variable respuesta sigue una distribución de Poisson cuya media depende de las covariables incluidas en el modelo.

7.2.1. Modelo de Poisson y medidas de bondad de ajuste

```

# Modelo de regresión de Poisson para goles locales
modelo_poisson <- glm(FTHG ~ HST + AST + HC + AC + HF + AF,
                      data   = train,
                      family = poisson)

summary(modelo_poisson)

```

```
##
## Call:
## glm(formula = FTHG ~ HST + AST + HC + AC + HF + AF, family = poisson,
##      data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.1091683  0.2947229  -0.370   0.7111
## HST          0.1978189  0.0193515  10.222 <2e-16 ***
## AST         -0.0324874  0.0229998  -1.413   0.1578
## HC          -0.0414868  0.0167643  -2.475   0.0133 *
## AC           0.0114668  0.0197600   0.580   0.5617
## HF          -0.0168539  0.0144281  -1.168   0.2428
## AF           0.0001021  0.0140184   0.007   0.9942
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 407.81  on 303  degrees of freedom
## Residual deviance: 286.51  on 297  degrees of freedom
## AIC: 900.36
##
## Number of Fisher Scoring iterations: 5

# Test de razón de verosimilitud (lmtest ya cargado en 2.1)
lrtest(modelo_poisson)

## Likelihood ratio test
##
## Model 1: FTHG ~ HST + AST + HC + AC + HF + AF
## Model 2: FTHG ~ 1
##   #Df LogLik Df Chisq Pr(>Chisq)
## 1    7 -443.18
## 2    1 -503.83 -6 121.3 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# R^2 de McFadden (pscl ya cargado en 2.1)
pR2(modelo_poisson)["McFadden"]

## fitting null model for pseudo-r2

## McFadden
## 0.1203806

# Predicción sobre el conjunto de prueba
pred_poisson <- predict(modelo_poisson, newdata = test, type = "response")

# Redondeamos al entero más cercano para obtener goles predichos
pred_poisson_redondeado <- round(pred_poisson)
```

```

# Comparación entre goles predichos y reales
comparacion_poisson <- data.frame(
  Real      = test$FTHG,
  Predicho  = pred_poisson_redondeado
)

# Error cuadrático medio (RMSE)
rmse_poisson <- sqrt(mean((test$FTHG - pred_poisson)^2))
round(rmse_poisson, 4)

```

```
## [1] 0.9438
```

```

# Error absoluto medio (MAE)
mae_poisson <- mean(abs(test$FTHG - pred_poisson))
round(mae_poisson, 4)

```

```
## [1] 0.7615
```

Los resultados del modelo de regresión de Poisson muestran que la variable más determinante para predecir los goles del equipo local es HST (tiros a puerta locales), con un coeficiente de 0.198 altamente significativo ($p < 2e-16$). Esto significa que cada tiro a puerta adicional del equipo local multiplica los goles esperados por $\exp(0.198) = 1.22$, es decir, aumenta un 22% los goles esperados. El resto de variables no resultan estadísticamente significativas al nivel del 5%, salvo HC (corners locales) con un efecto negativo leve (-0.041, $p = 0.013$), lo que podría indicar que los corners son consecuencia de ataques fallidos más que de dominio ofensivo real. El modelo obtiene un RMSE de 0.9438 y un MAE de 0.7615, lo que significa que en promedio el modelo se equivoca en 0.76 goles al predecir los goles del equipo local, resultado razonable dado que la media de goles locales es de 1.63. En conjunto, el modelo captura de forma aceptable el comportamiento medio de los goles locales, siendo los tiros a puerta la variable más determinante tal como confirmó el `summary()`. Por otro lado, el test de razón de verosimilitud verificó que con un p-valor de prácticamente 0 se rechaza la hipótesis nula, por lo que el modelo con todas las variables es significativamente mejor que el modelo nulo que tiene solo el intercepto y ninguna variable explicativo. A continuación, un R^2 de McFadden de 0.12 se considera un ajuste satisfactorio y esperado dentro del análisis estadístico deportivo de alto nivel. Este valor indica que variables poseen un poder explicativo real sobre la cantidad de goles anotados por el equipo local, permitiendo al modelo superar con creces a un modelo nulo o basado en el azar. No obstante, el hecho de que el 88% de la verosimilitud no sea capturada por estas métricas refleja la naturaleza del fútbol como un deporte de baja frecuencia de eventos y alta volatilidad, donde factores externos no cuantificables (como errores arbitrales, calidad individual en el remate o el azar del juego) mantienen un peso predominante. En conclusión, el modelo es una herramienta útil para identificar tendencias de peligro, aunque confirma que las estadísticas de flujo de juego por sí solas no pueden eliminar la incertidumbre inherente a un marcador final.

7.2.2. Ejemplos de predicción de goles

En este apartado de la subsección, se presentan 3 ejemplos de predicción del número de goles obtenidos por el equipo local en base a números de variables numéricas como los tiros a puerta, las faltas y los saques de esquina.

```

# Ejemplo de predicción 1: ¿cuántos goles marcaría el local si tiene
# 6 tiros a puerta, el visitante 3, 5 corners locales, 4 corners
# visitantes, 10 faltas locales y 12 faltas visitantes?
nuevo_partido1 <- data.frame(

```

```

HST = 6,
AST = 3,
HC  = 5,
AC  = 4,
HF  = 10,
AF  = 12
)

goles_predichos1 <- predict(modelo_poisson,
                           newdata = nuevo_partido1,
                           type     = "response")

round(goles_predichos1, 2)

```

```

##      1
## 1.92

```

```

# Ejemplo de predicción 2: partido muy dominado por el local
# 10 tiros a puerta locales, 1 visitante, 8 corners locales,
# 2 corners visitantes, 8 faltas locales y 15 faltas visitantes
nuevo_partido2 <- data.frame(
  HST = 10,
  AST = 1,
  HC  = 8,
  AC  = 2,
  HF  = 8,
  AF  = 15
)

goles_predichos2 <- predict(modelo_poisson,
                           newdata = nuevo_partido2,
                           type     = "response")

round(goles_predichos2, 2)

```

```

##      1
## 4.03

```

```

# Ejemplo de predicción 3: partido muy dominado por el visitante
# 2 tiros a puerta locales, 9 visitantes, 2 corners locales,
# 7 corners visitantes, 15 faltas locales y 6 faltas visitantes
nuevo_partido3 <- data.frame(
  HST = 2,
  AST = 9,
  HC  = 2,
  AC  = 7,
  HF  = 15,
  AF  = 6
)

goles_predichos3 <- predict(modelo_poisson,
                           newdata = nuevo_partido3,

```

```

type      = "response")

round(goles_predichos3, 2)

```

```

##      1
## 0.77

```

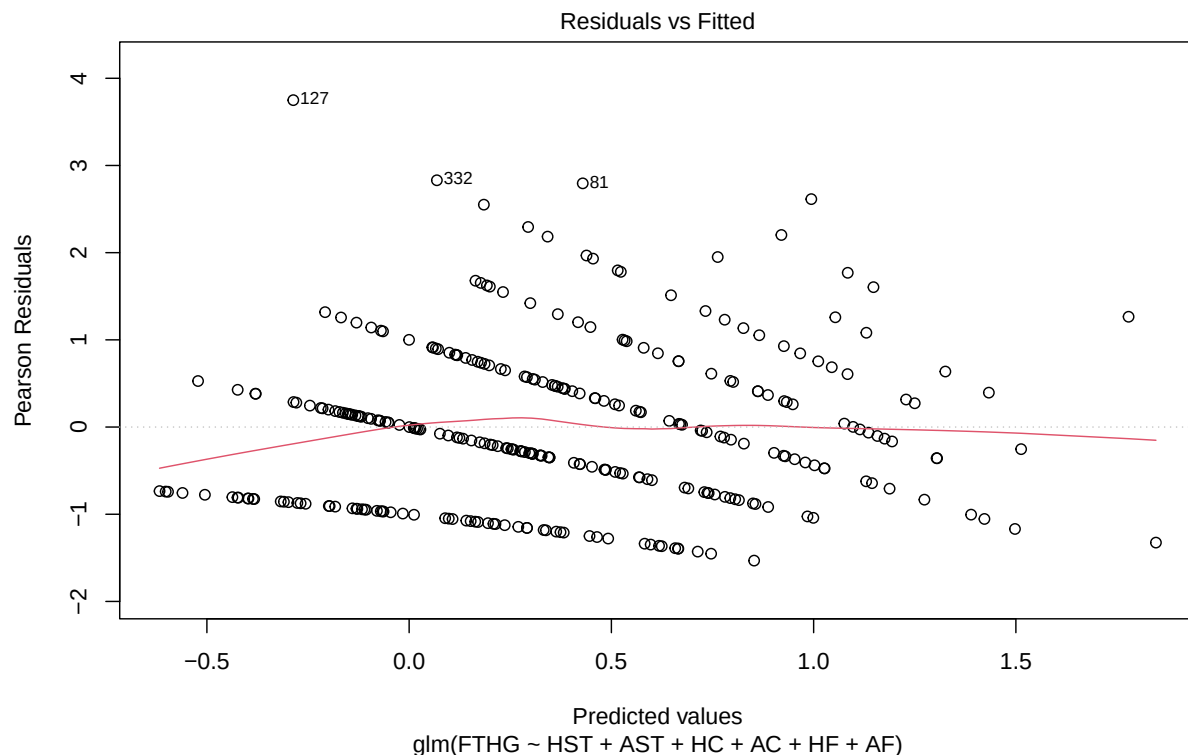
Los tres ejemplos de predicción ilustran el comportamiento del modelo de forma intuitiva y coherente con la realidad del fútbol. En el partido equilibrado con 6 tiros a puerta el modelo predice 1.92 goles esperados, valor cercano a la media real de 1.63 goles por partido. En el partido muy dominado por el local con 10 tiros a puerta el modelo predice 4.03 goles esperados, reflejando que un equipo con dominio ofensivo muy superior genera una expectativa de gol mucho mayor. En el partido muy dominado por el visitante con solo 2 tiros a puerta el modelo predice únicamente 0.77 goles esperados, indicando que un equipo con escasa presencia ofensiva tiene muy pocas probabilidades de marcar. En conjunto los tres escenarios confirman que el modelo captura correctamente la relación entre el dominio ofensivo y los goles esperados, siendo sensible a las diferencias en las estadísticas del partido de forma coherente con el análisis exploratorio de la sección 3.

7.2.3. Diagnóstico del modelo y validación por supuestos

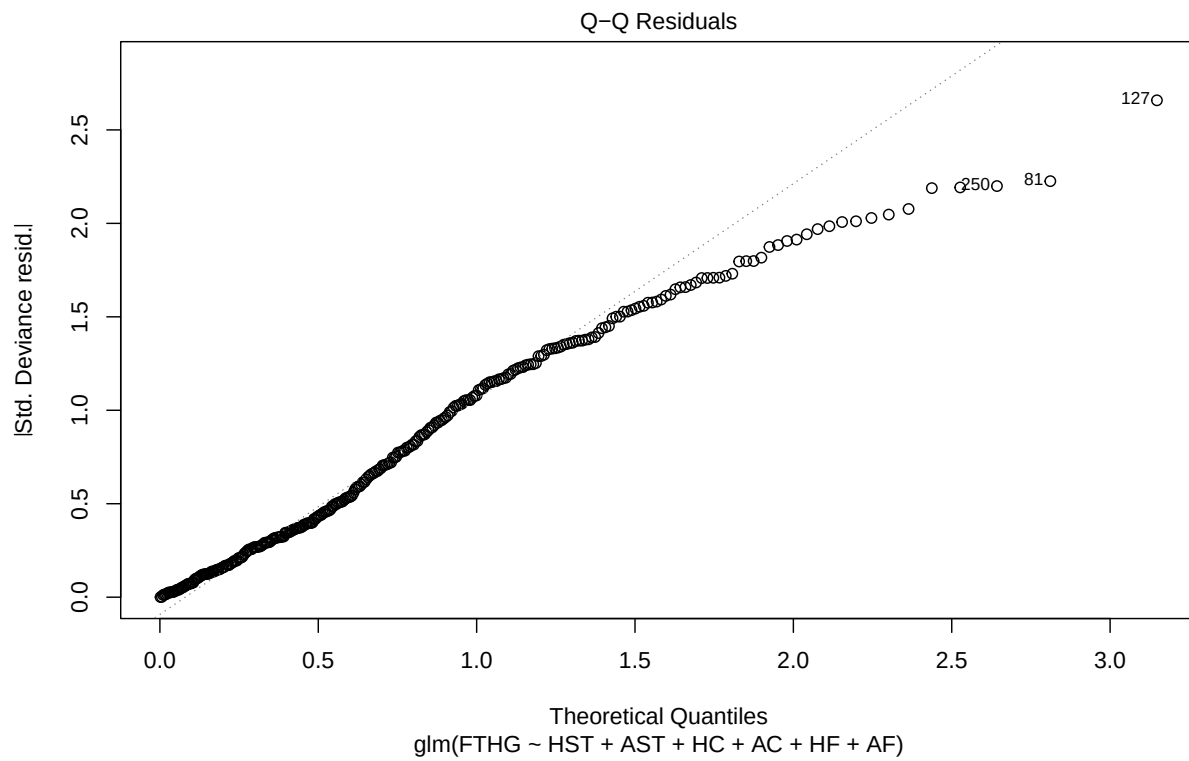
```

# Diagnóstico del modelo de Poisson
plot(modelo_poisson, which = 1) # Residuos vs valores ajustados

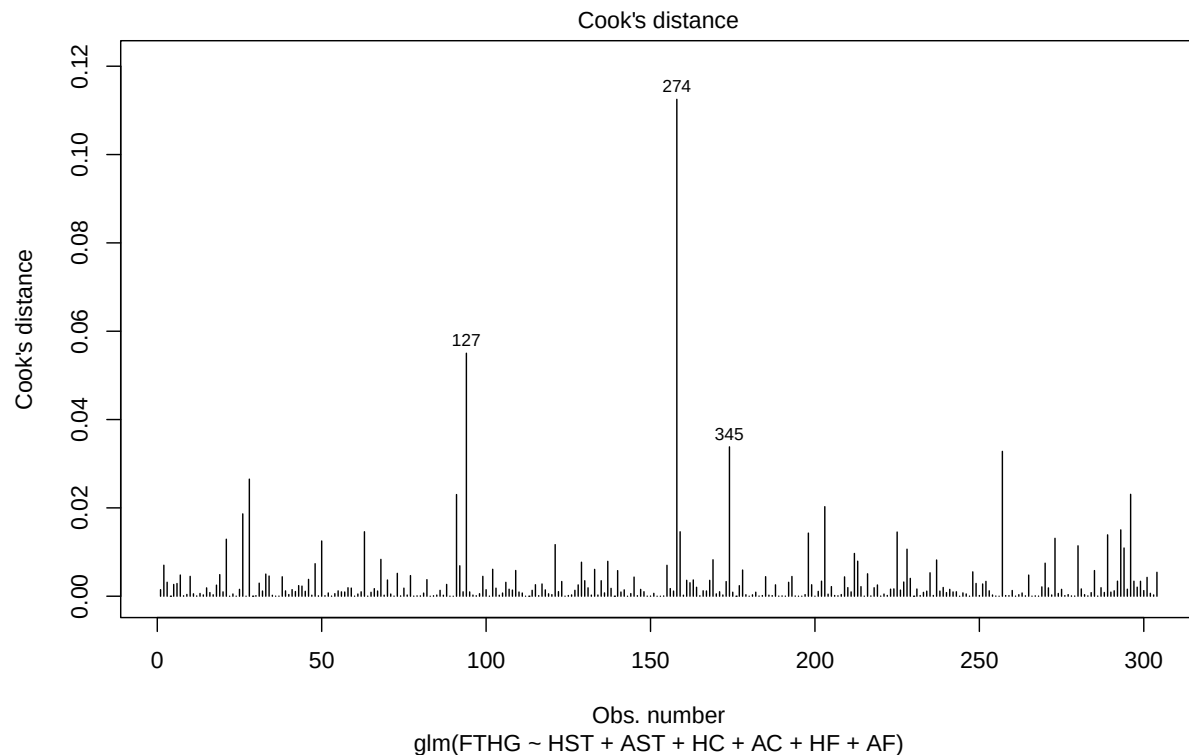
```



```
plot(modelo_poisson, which = 2) # Q-Q plot de residuos
```



```
plot(modelo_poisson, which = 4) # Distancia de Cook
```



```
# Observaciones influyentes (distancia de Cook > 4/n)
```

```
n <- nrow(train)
cook_poisson <- cooks.distance(modelo_poisson)
influyentes_poisson <- which(cook_poisson > 4/n)
cat("Número de observaciones influyentes:", length(influyentes_poisson), "\n")
```

```
## Número de observaciones influyentes: 15
```

```
cat("Índices:", influyentes_poisson, "\n")
```

```
## Índices: 26 28 63 91 94 158 159 174 198 203 225 257 289 293 296
```

```
# Observación MÁS influyente (derivada, robusta): partido correspondiente
```

```
pos_max <- which.max(cook_poisson)
train[pos_max, ] # = fila 274 del dataset: Brighton 3-3 Brentford
```

```
##      Date Time HomeTeam AwayTeam FTHG FTAG FTR HTHG HTAG HTR Referee HS
## 274 2023-04-01 15:00 Brighton Brentford 3 3 D 2 2 D M Oliver 33
##      AS HST AST HF AF HC AC HY AY HR AR TotalGoals HomeWin Over2_5 GoalDiff
## 274 7 15 5 16 10 14 0 2 1 0 0 6 0 1 0
##      ShotsDiff ShotsTargetDiff EfficiencyHome EfficiencyAway EfficiencyDiff
## 274 26 10 0.2 0.6 -0.4
##      PrecisionHome PrecisionAway TotalShots ShotRatioHome RedCard
## 274 0.4545455 0.7142857 40 0.825 0
```

El gráfico de residuos de Pearson frente a valores ajustados muestra los residuos distribuidos de forma relativamente aleatoria alrededor de cero, sin patrones sistemáticos claros, lo que respalda la idoneidad del modelo de Poisson; los residuos más altos corresponden a partidos con un número de goles inusualmente alto respecto a lo esperado. El Q-Q plot indica que los residuos siguen razonablemente bien la distribución teórica en la zona central, con desviaciones en los extremos, algo habitual en modelos de conteo con partidos de muchos goles.

La distancia de Cook (umbral $4/n$) identifica 15 observaciones influyentes sobre las 304 de entrenamiento (4.9%), en las posiciones 26, 28, 63, 91, 94, 158, 159, 174, 198, 203, 225, 257, 289, 293 y 296 del conjunto de entrenamiento, un porcentaje razonable que no compromete la fiabilidad del modelo. La observación MÁS influyente es la posición 158 del entrenamiento, que corresponde a la fila 274 del dataset: el partido Brighton 3-3 Brentford (1 de abril de 2023), en el que el Brighton realizó 33 tiros (15 a puerta) y aun así solo empató. Este encuentro desafía al modelo porque un dominio ofensivo extremo no se tradujo en victoria, generando un residuo elevado que influye de forma desproporcionada en la estimación de los coeficientes: un ejemplo claro de cómo la eficiencia puntual y el azar pueden distorsionar la relación entre estadísticas ofensivas y resultado.

7.3. Regresión logística binomial: predicción de victoria local

En esta subsección se ajusta un modelo de regresión logística binomial para predecir si el equipo local gana el partido ($\text{HomeWin} = 1$) o no ($\text{HomeWin} = 0$). La regresión logística es la elección natural para modelizar variables respuesta binarias, estimando la probabilidad de victoria local en función de las estadísticas del partido. Al igual que en el modelo de Poisson, se evalúa la significatividad global mediante el test de razón de verosimilitud y la capacidad explicativa mediante el R^2 de McFadden.

Cabe destacar una decisión clave de selección de variables: los clasificadores emplean únicamente las estadísticas brutas del partido (HST, AST, HC, AC, HF, AF). Se excluyen las variables de eficiencia (EfficiencyHome, EfficiencyAway) porque se calculan como goles / tiros a puerta y, por tanto, incorporan el resultado que se pretende predecir: incluirlas inflaría artificialmente el accuracy (fuga de información o data leakage). Se excluyen también las diferencias derivadas como ShotsTargetDiff por ser combinaciones lineales exactas de variables ya incluidas (HST - AST), lo que provoca colinealidad perfecta y matrices de covarianzas singulares en el QDA.

7.3.1. Modelo logístico y medidas de bondad de ajuste

```
# Modelo de regresión logística binomial
modelo_logistico <- glm(HomeWin ~ HST + AST + HC + AC + HF + AF,
  data = train,
  family = binomial)

summary(modelo_logistico)
```

```
##
## Call:
## glm(formula = HomeWin ~ HST + AST + HC + AC + HF + AF, family = binomial,
##      data = train)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.67555    0.86317  -0.783  0.43384
## HST          0.33808    0.06321   5.348 8.88e-08 ***
## AST         -0.42084    0.07515  -5.600 2.15e-08 ***
```

```
## HC          -0.03977    0.04861   -0.818   0.41328
## AC           0.14686    0.05609    2.618   0.00883 **
## HF          -0.01042    0.04120   -0.253   0.80033
## AF           0.02218    0.03920    0.566   0.57149
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 421.22  on 303  degrees of freedom
## Residual deviance: 344.94  on 297  degrees of freedom
## AIC: 358.94
##
## Number of Fisher Scoring iterations: 4
```

```
# Test de razón de verosimilitud
lrtest(modelo_logistico)
```

```
## Likelihood ratio test
##
## Model 1: HomeWin ~ HST + AST + HC + AC + HF + AF
## Model 2: HomeWin ~ 1
##   #Df LogLik Df  Chisq Pr(>Chisq)
## 1    7 -172.47
## 2    1 -210.61 -6  76.281  2.091e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# R^2 de McFadden
pR2(modelo_logistico)["McFadden"]
```

```
## fitting null model for pseudo-r2
```

```
## McFadden
## 0.181094
```

El test de razón de verosimilitud confirma la significatividad global del modelo (Chi-cuadrado = 76.3, $p < 0.001$ frente al modelo nulo) y el R^2 de McFadden (0.181) se sitúa en el rango habitual de los modelos logísticos bien especificados sobre fenómenos ruidosos.

7.3.2. Ejemplos de predicción e interpretación de Odds Ratios

```
# Predicción sobre el conjunto de prueba
prob_logistico <- predict(modelo_logistico, newdata = test, type = "response")

# Clasificación con umbral 0.5 (se evaluará en la subsección 7.6)
pred_logistico <- ifelse(prob_logistico > 0.5, 1, 0)
```

Ejemplos de predicción: tres escenarios con grados de dominio distintos

```

# Escenario 1: partido equilibrado
p_equilibrado <- predict(modelo_logistico,
                        newdata = data.frame(HST = 5, AST = 4, HC = 5,
                                              AC = 5, HF = 11, AF = 11),
                        type = "response")

# Escenario 2: dominio claro del equipo local
p_dominio_local <- predict(modelo_logistico,
                          newdata = data.frame(HST = 15, AST = 3, HC = 14,
                                                AC = 7, HF = 8, AF = 11),
                          type = "response")

# Escenario 3: dominio claro del equipo visitante
p_dominio_visit <- predict(modelo_logistico,
                          newdata = data.frame(HST = 2, AST = 9, HC = 2,
                                                AC = 7, HF = 15, AF = 6),
                          type = "response")

cat("Partido equilibrado (5 tiros a puerta vs 4):", round(p_equilibrado * 100, 1), "%\n")

## Partido equilibrado (5 tiros a puerta vs 4): 49.9 %

cat("Dominio local (15 tiros a puerta vs 3):      ", round(p_dominio_local * 100, 1), "%\n")

## Dominio local (15 tiros a puerta vs 3):      97.7 %

cat("Dominio visitante (2 tiros a puerta vs 9):  ", round(p_dominio_visit * 100, 1), "%\n")

## Dominio visitante (2 tiros a puerta vs 9):   5.4 %

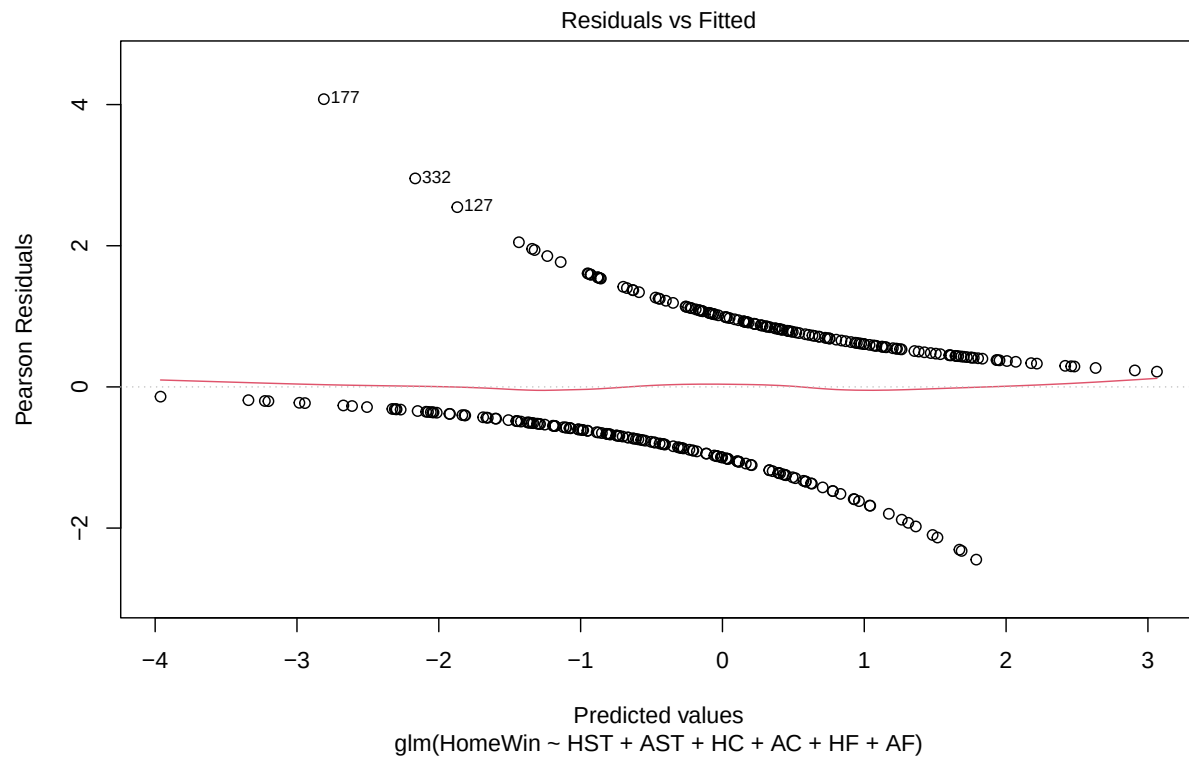
```

Los coeficientes se interpretan en escala log-odds. Las dos variables determinantes son los tiros a puerta de ambos equipos: HST presenta un coeficiente de 0.338 ($p < 0.001$), de modo que cada tiro a puerta local adicional multiplica las odds de victoria por $\exp(0.338) = 1.40$, mientras que AST presenta un coeficiente de -0.421 ($p < 0.001$), reduciendo las odds en un factor $\exp(-0.421) = 0.66$ por cada tiro a puerta visitante. Entre el resto de variables destaca el efecto positivo y significativo de los corners visitantes (AC, coeficiente 0.147, $p < 0.01$), un resultado a primera vista contraintuitivo: más corners del rival se asocia a mayor probabilidad de victoria local. La explicación más plausible es la dinámica del marcador: el equipo que va por detrás fuerza más saques de esquina en su intento de remontada, de modo que acumular corners refleja ir perdiendo más que dominar. Este hallazgo es coherente con el efecto negativo de los corners locales (HC) detectado en el modelo de Poisson. Las faltas y los corners locales no resultan significativos.

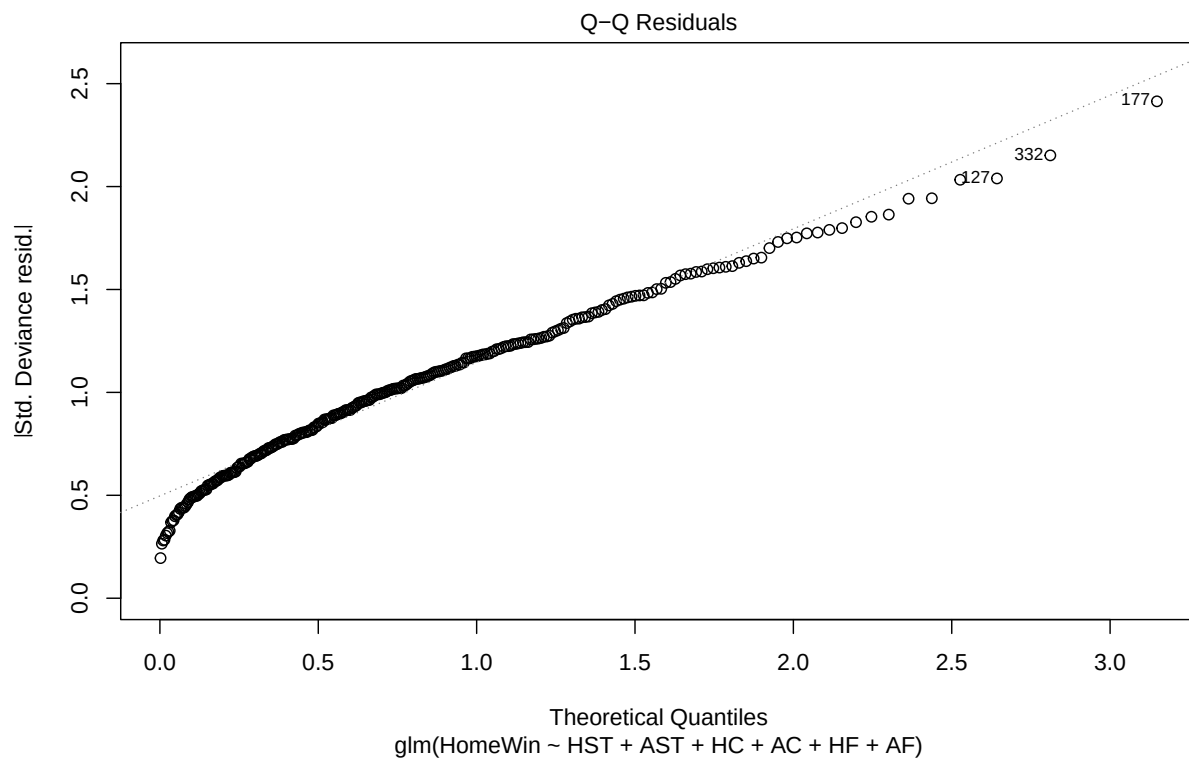
Los ejemplos de predicción ilustran el modelo en tres escenarios: en un partido completamente igualado la probabilidad de victoria local ronda el 50% (49.9%), con dominio local claro (15 tiros a puerta frente a 3) asciende al 97.7%, y con dominio visitante se desploma al 5.4%. Cabe señalar que este modelo no incluye un término explícito de ventaja de campo: las estadísticas del propio partido ya absorben ese efecto, a diferencia del Dixon-Coles de la sección 5, que lo aísla como parámetro.

7.3.3. Diagnóstico del modelo y validación por supuestos

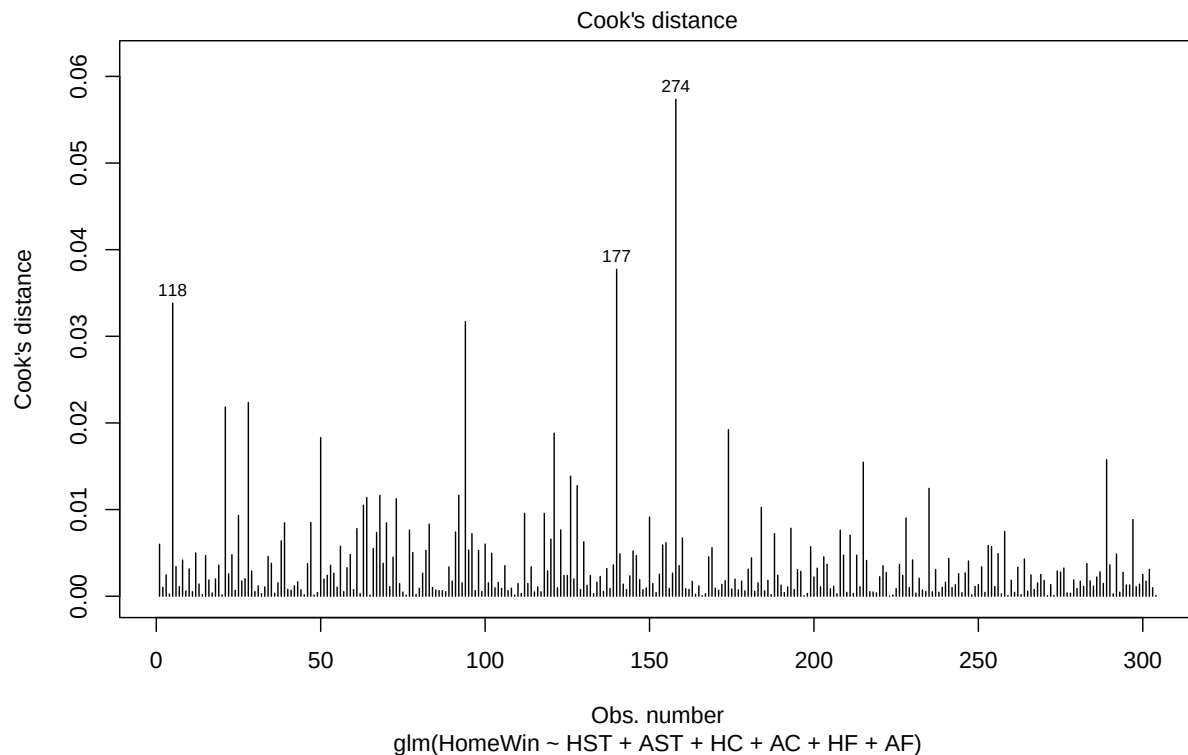
```
# Diagn sis del modelo log stico
plot(modelo_logistico, which = 1) # Residuos vs valores ajustados
```



```
plot(modelo_logistico, which = 2) # Q-Q plot de residuos
```



```
plot(modelo_logistico, which = 4) # Distancia de Cook
```



```
# Observaciones influyentes (distancia de Cook > 4/n)
cook_logistico <- cooks.distance(modelo_logistico)
influyentes_log <- which(cook_logistico > 4/nrow(train))
cat("Número de observaciones influyentes:", length(influyentes_log), "\n")
```

```
## Número de observaciones influyentes: 12
```

La diagnosis sigue el mismo esquema que en el modelo de Poisson y detecta 12 partidos influyentes (un 3.9% del entrenamiento), proporción reducida que confirma que la estimación de los coeficientes es robusta y no está dirigida por un puñado de encuentros atípicos.

7.4. Análisis discriminante lineal (LDA)

En esta subsección se aplica el análisis discriminante lineal para clasificar los partidos según victoria local o no victoria. El LDA busca la combinación lineal de las variables predictoras que mejor separa ambas clases, asumiendo normalidad multivariante y matrices de covarianzas iguales entre grupos. Se utiliza la misma variable respuesta binaria (HomeWin) que en la regresión logística para que ambos modelos sean directamente comparables en la subsección 7.6.

(MASS ya cargado en 2.1)

```
# Ajuste del modelo LDA
modelo_lda <- lda(HomeWin ~ HST + AST + HC + AC + HF + AF,
                  data = train)

modelo_lda
```

```
## Call:
## lda(HomeWin ~ HST + AST + HC + AC + HF + AF, data = train)
##
## Prior probabilities of groups:
##      0      1
## 0.5131579 0.4868421
##
## Group means:
##      HST      AST      HC      AC      HF      AF
## 0 4.128205 4.641026 5.500000 4.608974 10.67308 10.82051
## 1 5.750000 3.094595 5.810811 4.331081 10.62162 11.05405
##
## Coefficients of linear discriminants:
##      LD1
## HST  0.319234491
## AST -0.380376628
## HC  -0.042145284
## AC   0.132870943
## HF  -0.009339257
## AF   0.024354581

# Predicción sobre el conjunto de prueba
pred_lda_obj <- predict(modelo_lda, newdata = test)

# Clases predichas
pred_lda <- pred_lda_obj$class

# Probabilidades a posteriori de victoria local (clase 1)
prob_lda <- pred_lda_obj$posterior[, "1"]

head(prob_lda)

##      1      3      6      8     15     17
## 0.4619736 0.5514750 0.7437432 0.6121524 0.3644427 0.5720895
```

Los coeficientes del discriminante lineal indican el peso de cada variable en la separación entre victorias y no victorias locales: las variables con mayor coeficiente en valor absoluto son las que más contribuyen a discriminar entre ambos grupos. Las probabilidades a posteriori permiten, además de clasificar, cuantificar la confianza del modelo en cada predicción, lo que resulta esencial para la construcción de las curvas ROC de la subsección 7.6.

7.5. Clasificación mediante KNN

En esta subsección se aplica el algoritmo de los k vecinos más cercanos. A diferencia de los modelos anteriores, KNN es un método no paramétrico: no asume ninguna distribución sobre los datos y clasifica cada partido según la clase mayoritaria de sus k vecinos más próximos. Dado que el algoritmo se basa en distancias, es imprescindible escalar previamente las variables para que ninguna domine el cálculo por su magnitud. Se escala el conjunto de prueba con la media y desviación típica del conjunto de entrenamiento para evitar fuga de información.

(class ya cargado en 2.1)

```

# Variables predictorias (las mismas que en logística y LDA)
vars_knn <- c("HST", "AST", "HC", "AC", "HF", "AF")

# Escalado del conjunto de entrenamiento
train_knn <- scale(train[, vars_knn])

# Escalado del conjunto de prueba con los parámetros del entrenamiento
test_knn <- scale(test[, vars_knn],
                  center = attr(train_knn, "scaled:center"),
                  scale = attr(train_knn, "scaled:scale"))

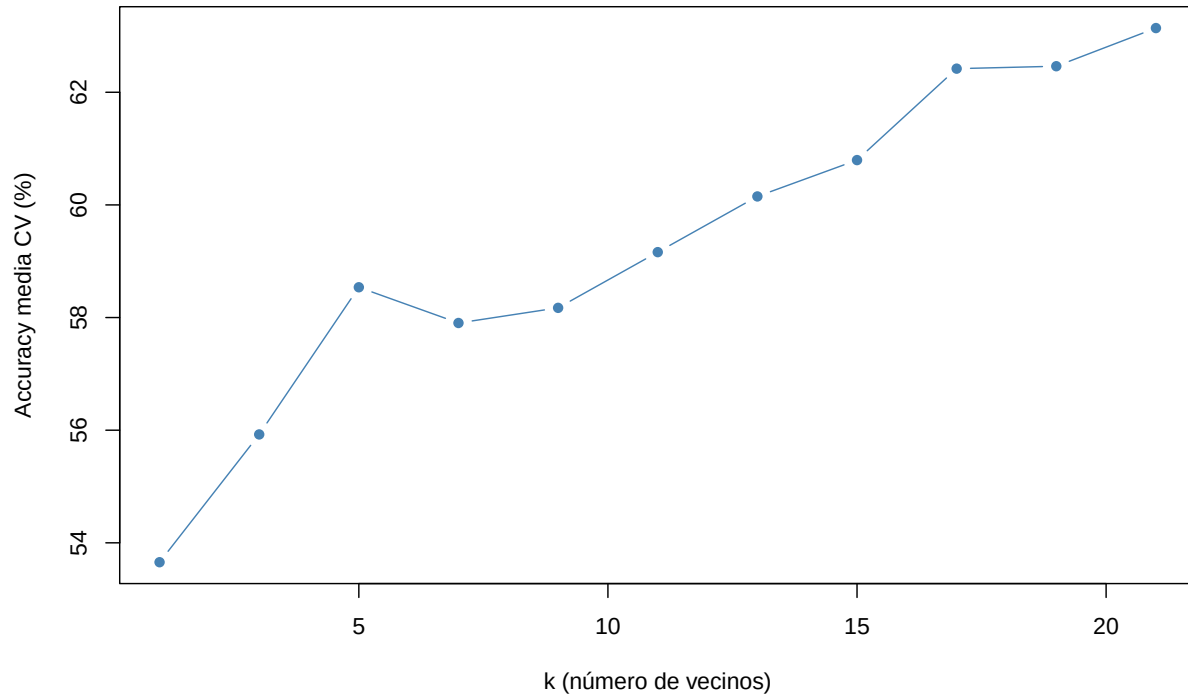
# Selección de k mediante validación cruzada 10-fold sobre el ENTRENAMIENTO
# (elegir k mirando el test contaminaría la evaluación final)
set.seed(123)
ks <- seq(1, 21, by = 2) # solo valores impares para evitar empates
folds_k <- sample(rep(1:10, length.out = nrow(train)))
write.csv(data.frame(fold = folds_k), "folds_knn.csv", row.names = FALSE) # PUENTE de folds
acc_por_k <- numeric(length(ks))

for (j in seq_along(ks)) {
  acc_fold <- numeric(10)
  for (f in 1:10) {
    tr_f <- train[folds_k != f, ]
    te_f <- train[folds_k == f, ]
    tr_s <- scale(tr_f[, vars_knn])
    te_s <- scale(te_f[, vars_knn],
                  center = attr(tr_s, "scaled:center"),
                  scale = attr(tr_s, "scaled:scale"))
    p <- knn(tr_s, te_s, cl = tr_f$HomeWin, k = ks[j])
    acc_fold[f] <- mean(p == te_f$HomeWin)
  }
  acc_por_k[j] <- mean(acc_fold)
}

# Gráfico: accuracy de validación cruzada frente a k
plot(ks, acc_por_k * 100, type = "b", pch = 16, col = "steelblue",
     xlab = "k (número de vecinos)",
     ylab = "Accuracy media CV (%)",
     main = "Selección de k mediante validación cruzada")

```

Selección de k mediante validación cruzada



```
k_optimo <- ks[which.max(acc_por_k)]
cat("k óptimo:", k_optimo,
    "(accuracy CV =", round(max(acc_por_k) * 100, 1), "%)\n")
```

```
## k óptimo: 21 (accuracy CV = 63.1 %)
```

```
# Clasificación final con el k óptimo seleccionado
set.seed(123)
pred_knn <- knn(train = train_knn,
               test  = test_knn,
               cl    = train$HomeWin,
               k     = k_optimo,
               prob  = TRUE)

# Probabilidad de victoria local (clase 1) para las curvas ROC
prob_ganadora <- attr(pred_knn, "prob")
prob_knn <- ifelse(pred_knn == 1, prob_ganadora, 1 - prob_ganadora)

table(pred_knn)
```

```
## pred_knn
## 0 1
## 43 33
```

En lugar de fijar k a priori, se selecciona mediante validación cruzada 10-fold sobre el conjunto de entrenamiento, explorando únicamente valores impares (que evitan empates en la votación entre las dos clases).

Es fundamental que esta búsqueda se realice exclusivamente sobre el entrenamiento: elegir k observando el accuracy del test contaminaría la evaluación final, que debe medirse sobre datos completamente ajenos a cualquier decisión del modelado. El gráfico muestra el compromiso clásico del KNN: # valores muy pequeños de k producen fronteras ruidosas y sobreajustadas, mientras que valores muy grandes las suavizan en exceso y pierden capacidad discriminante. En este caso el óptimo se alcanza en $k = 21$ (accuracy de validación cruzada del 63.1%), un valor elevado que indica que la señal local es débil y el algoritmo necesita promediar muchos vecinos para estabilizar la decisión. La distribución final de las predicciones sobre el test (43 no victorias frente a 33 victorias) es coherente con el reparto real de clases (40 y 36).

7.6. Comparación de modelos y validación cruzada

En esta subsección se evalúan y comparan los cuatro clasificadores: regresión logística, LDA, QDA y KNN. El QDA se incluye únicamente en esta comparación final: es una variante del LDA que elimina el supuesto de igualdad de matrices de covarianzas, por lo que contrastar ambos permite validar empíricamente dicho supuesto. La comparación se realiza mediante matrices de confusión, accuracy sobre el conjunto de prueba y curvas ROC con su área bajo la curva (AUC), que mide la capacidad discriminante del modelo con independencia del umbral de clasificación.

```
# Ajuste del QDA (solo para comparación, sin sección propia)
modelo_qda <- qda(HomeWin ~ HST + AST + HC + AC + HF + AF,
                  data = train)
```

```
pred_qda_obj <- predict(modelo_qda, newdata = test)
pred_qda <- pred_qda_obj$class
prob_qda <- pred_qda_obj$posterior[, "1"]
```

```
# Matrices de confusión (Real en filas, Predicho en columnas)
cat("Matriz de confusión - Regresión logística:\n")
```

```
## Matriz de confusión - Regresión logística:
```

```
mc_log <- table(Real = test$HomeWin, Predicho = pred_logistico)
print(mc_log)
```

```
##      Predicho
## Real  0  1
##      0 32  8
##      1  9 27
```

```
cat("\nMatriz de confusión - LDA:\n")
```

```
##
## Matriz de confusión - LDA:
```

```
mc_lda <- table(Real = test$HomeWin, Predicho = pred_lda)
print(mc_lda)
```

```
##      Predicho
## Real  0  1
##      0 32  8
##      1  9 27
```

```
cat("\nMatriz de confusión - QDA:\n")
```

```
##  
## Matriz de confusión - QDA:
```

```
mc_qda <- table(Real = test$HomeWin, Predicho = pred_qda)  
print(mc_qda)
```

```
##      Predicho  
## Real  0  1  
##      0 31  9  
##      1 12 24
```

```
cat("\nMatriz de confusión - KNN:\n")
```

```
##  
## Matriz de confusión - KNN:
```

```
mc_knn <- table(Real = test$HomeWin, Predicho = pred_knn)  
print(mc_knn)
```

```
##      Predicho  
## Real  0  1  
##      0 28 12  
##      1 15 21
```

```
# Accuracy de cada modelo  
acc_log <- sum(diag(mc_log)) / sum(mc_log)  
acc_lda <- sum(diag(mc_lda)) / sum(mc_lda)  
acc_qda <- sum(diag(mc_qda)) / sum(mc_qda)  
acc_knn <- sum(diag(mc_knn)) / sum(mc_knn)  
  
tabla_accuracy <- data.frame(  
  Modelo = c("Logística", "LDA", "QDA", "KNN"),  
  Accuracy = round(c(acc_log, acc_lda, acc_qda, acc_knn) * 100, 1)  
)  
tabla_accuracy[order(-tabla_accuracy$Accuracy), ]
```

```
##      Modelo Accuracy  
## 1 Logística      77.6  
## 2      LDA      77.6  
## 3      QDA      72.4  
## 4      KNN      64.5
```

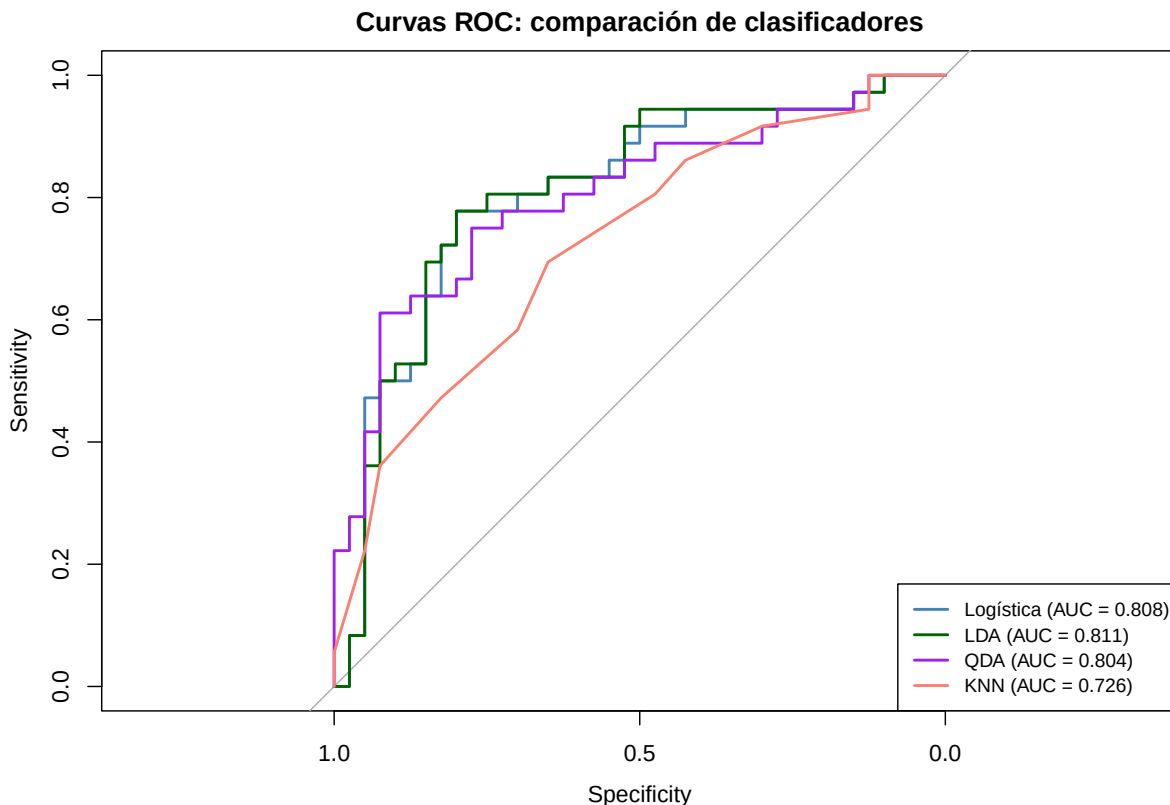
Las matrices de confusión revelan que los errores de los modelos lineales están razonablemente equilibrados entre ambos tipos: la logística, por ejemplo, comete 8 falsos positivos (no victorias clasificadas como victoria) y 9 falsos negativos sobre 76 partidos, sin sesgo sistemático hacia ninguna clase. Ese equilibrio es deseable: un modelo que acertase mucho una clase a costa de ignorar la otra tendría un accuracy engañoso.

```

# Curvas ROC y AUC (pROC ya cargado en 2.1)
roc_log <- roc(test$HomeWin, prob_logistico)
roc_lda <- roc(test$HomeWin, prob_lda)
roc_qda <- roc(test$HomeWin, prob_qda)
roc_knn <- roc(test$HomeWin, prob_knn)

# Gráfica comparativa
plot(roc_log, col = "steelblue", lwd = 2,
     main = "Curvas ROC: comparación de clasificadores")
lines(roc_lda, col = "darkgreen", lwd = 2)
lines(roc_qda, col = "purple", lwd = 2)
lines(roc_knn, col = "salmon", lwd = 2)
legend("bottomright",
      legend = c(paste0("Logística (AUC = ", round(auc(roc_log), 3), ")"),
                 paste0("LDA (AUC = ", round(auc(roc_lda), 3), ")"),
                 paste0("QDA (AUC = ", round(auc(roc_qda), 3), ")"),
                 paste0("KNN (AUC = ", round(auc(roc_knn), 3), ")")),
      col = c("steelblue", "darkgreen", "purple", "salmon"),
      lwd = 2, cex = 0.8)

```



Las curvas ROC ofrecen una lectura independiente del umbral de clasificación: la logística (AUC = 0.808), el LDA (0.811) y el QDA (0.804) muestran capacidades discriminantes prácticamente idénticas y notablemente altas, ordenando correctamente cuatro de cada cinco pares victoria/no victoria, mientras que el KNN queda claramente por detrás. Que el QDA recupere terreno respecto a su accuracy indica que sus probabilidades

están bien ordenadas aunque el umbral de 0.5 no le favorezca. Valores de AUC en torno a 0.80 son sólidos para un fenómeno con tanto azar como el resultado de un partido de fútbol.

```
# Validación cruzada 10-fold (accuracy media de cada modelo)
set.seed(123)
k_folds <- 10
folds <- sample(rep(1:k_folds, length.out = nrow(datos)))
write.csv(data.frame(fold = folds), "folds_cv.csv", row.names = FALSE) # PUENTE de folds

acc_cv <- matrix(0, nrow = k_folds, ncol = 4)
colnames(acc_cv) <- c("Logística", "LDA", "QDA", "KNN")

for (f in 1:k_folds) {

  tr <- datos[folds != f, ]
  te <- datos[folds == f, ]

  # Logística
  m1 <- glm(HomeWin ~ HST + AST + HC + AC + HF + AF,
            data = tr, family = binomial)
  p1 <- ifelse(predict(m1, te, type = "response") > 0.5, 1, 0)
  acc_cv[f, 1] <- mean(p1 == te$HomeWin)

  # LDA
  m2 <- lda(HomeWin ~ HST + AST + HC + AC + HF + AF, data = tr)
  acc_cv[f, 2] <- mean(predict(m2, te)$class == te$HomeWin)

  # QDA
  m3 <- qda(HomeWin ~ HST + AST + HC + AC + HF + AF, data = tr)
  acc_cv[f, 3] <- mean(predict(m3, te)$class == te$HomeWin)

  # KNN (escalando dentro de cada fold)
  tr_s <- scale(tr[, vars_knn])
  te_s <- scale(te[, vars_knn],
                center = attr(tr_s, "scaled:center"),
                scale = attr(tr_s, "scaled:scale"))
  p4 <- knn(tr_s, te_s, cl = tr$HomeWin, k = k_optimo)
  acc_cv[f, 4] <- mean(p4 == te$HomeWin)
}

round(colMeans(acc_cv) * 100, 1)
```

## Logística	LDA	QDA	KNN
## 68.7	69.5	66.8	65.8

La regresión logística y el LDA empatan como mejores clasificadores con un 77.6% de accuracy en el conjunto de prueba, resultado esperable dado que ambos construyen fronteras de decisión lineales y convergen a soluciones casi idénticas con estos datos. El QDA queda por detrás (72.4%), lo que valida empíricamente el supuesto de igualdad de covarianzas del LDA: la flexibilidad adicional del QDA no aporta y sus parámetros extra sobreajustan, por lo que el modelo más simple es preferible por el principio de parsimonia. El KNN queda en último lugar incluso tras seleccionar k por validación cruzada, aunque supera con claridad al clasificador trivial que predeciría siempre la clase mayoritaria (52.6%): su naturaleza local sufre en un espacio de seis dimensiones con poco más de 300 partidos. La validación cruzada ofrece estimaciones más

conservadoras (logística y LDA en torno al 70%, con QDA y KNN algo por debajo y muy próximos entre sí): la diferencia respecto al test refleja la elevada varianza de evaluar sobre solo 76 partidos, y queda dentro de la desviación típica reportada por la propia validación cruzada. En conjunto, un accuracy máximo cercano al 78% es un resultado sólido y honesto para este problema: el fútbol presenta un techo de predictibilidad bajo, y valores muy superiores serían indicio de fuga de información más que de un buen modelo.

8. ¿Qué patrones de juego existen entre equipos?

En esta sección se aplican técnicas de aprendizaje no supervisado para identificar perfiles de juego entre los 20 equipos de la competición. A diferencia de las secciones anteriores, aquí no se predice ninguna variable: el objetivo es descubrir estructura oculta en los datos. Para ello se agregan las estadísticas de cada equipo a lo largo de la temporada, se reduce la dimensionalidad mediante componentes principales y se segmentan los equipos mediante K-means.

8.1. Reducción de dimensionalidad mediante PCA

En esta subsección se construye el perfil táctico medio de cada equipo (promediando sus estadísticas como local y como visitante) y se aplica el análisis de componentes principales sobre las variables numéricas tácticas, estandarizadas previamente para que todas contribuyan en igualdad de condiciones.

```
# Perfil táctico medio de cada equipo (local y visitante)
perfil_local <- aggregate(cbind(GolesFavor = FTHG,
                                GolesContra = FTAG,
                                Tiros = HS,
                                TirosPuerta = HST,
                                Corners = HC,
                                Faltas = HF) ~ HomeTeam,
                          data = datos, FUN = mean)

perfil_visit <- aggregate(cbind(GolesFavor = FTAG,
                                GolesContra = FTHG,
                                Tiros = AS,
                                TirosPuerta = AST,
                                Corners = AC,
                                Faltas = AF) ~ AwayTeam,
                          data = datos, FUN = mean)

# Promedio de ambos contextos
perfil_equipos <- data.frame(
  Equipo = perfil_local$HomeTeam,
  (perfil_local[, -1] + perfil_visit[, -1]) / 2
)

# PCA sobre las variables tácticas estandarizadas
pca_equipos <- prcomp(perfil_equipos[, -1], scale. = TRUE)

# Varianza explicada
summary(pca_equipos)
```

```
## Importance of components:
```

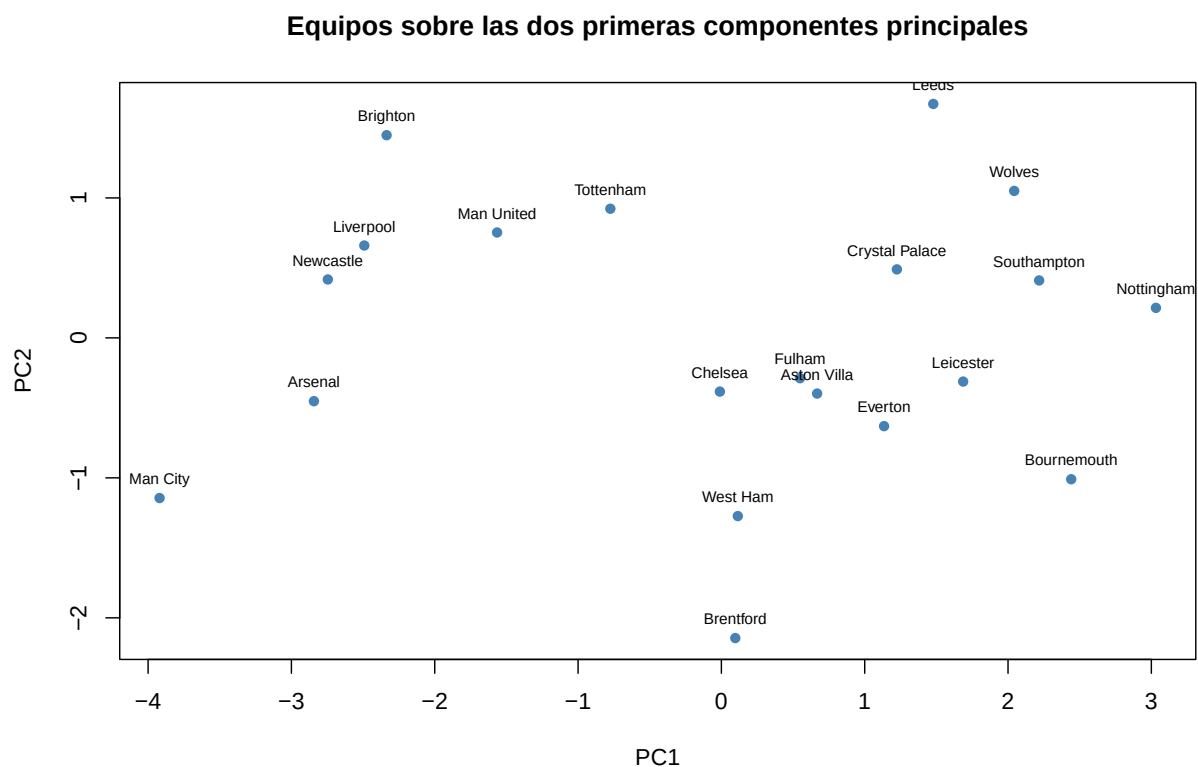
```
##           PC1      PC2      PC3      PC4      PC5      PC6
```

```
## Standard deviation      2.0330 0.9763 0.70746 0.49874 0.36623 0.17383
## Proportion of Variance 0.6889 0.1589 0.08342 0.04146 0.02235 0.00504
## Cumulative Proportion  0.6889 0.8477 0.93115 0.97261 0.99496 1.00000
```

```
# Cargas de las dos primeras componentes
round(pca_equipos$rotation[, 1:2], 3)
```

```
##           PC1    PC2
## GolesFavor -0.441 -0.015
## GolesContra 0.384  0.283
## Tiros      -0.461  0.297
## TirosPuerta -0.458  0.173
## Corners    -0.423  0.249
## Faltas      0.240  0.860
```

```
# Representación de los equipos sobre las dos primeras componentes
plot(pca_equipos$x[, 1], pca_equipos$x[, 2],
     main = "Equipos sobre las dos primeras componentes principales",
     xlab = "PC1", ylab = "PC2", pch = 16, col = "steelblue")
text(pca_equipos$x[, 1], pca_equipos$x[, 2],
     labels = perfil_equipos$Equipo, cex = 0.7, pos = 3)
```



Las dos primeras componentes condensan el 84.8% de la variabilidad táctica de la liga (un 68.9% la PC1 y un 15.9% la PC2), por lo que el plano PC1-PC2 constituye un resumen muy fiel de los perfiles de juego. Las

cargas identifican la PC1 como un eje de dominio ofensivo global: contrapone la producción ofensiva (tiros, tiros a puerta, corners y goles a favor, todas con cargas del mismo signo) a los goles encajados, de modo que un extremo del eje combina mucho ataque con poca concesión defensiva. La PC2 está dominada por las faltas (carga 0.86) y puede leerse como un eje de fricción o agresividad, independiente del dominio ofensivo. La proyección de los equipos sobre ambas componentes anticipa visualmente los grupos que el algoritmo K-means formalizará en la siguiente subsección.

8.2. Identificación de grupos mediante K-means

En esta subsección se segmentan los equipos en tres grupos mediante K-means sobre las variables tácticas estandarizadas, y se valida la calidad de la partición mediante el análisis de silueta, que mide para cada equipo si está bien asignado a su cluster (valores próximos a 1) o en la frontera entre grupos (valores próximos a 0).

(cluster ya cargado en 2.1)

```
# Estandarización de las variables tácticas
perfil_escalado <- scale(perfil_equipos[, -1])

# K-means con 3 clusters
set.seed(123)
kmeans_equipos <- kmeans(perfil_escalado, centers = 3, nstart = 25)

# Asignación de cada equipo a su cluster
data.frame(Equipo = perfil_equipos$Equipo,
           Cluster = kmeans_equipos$cluster)
```

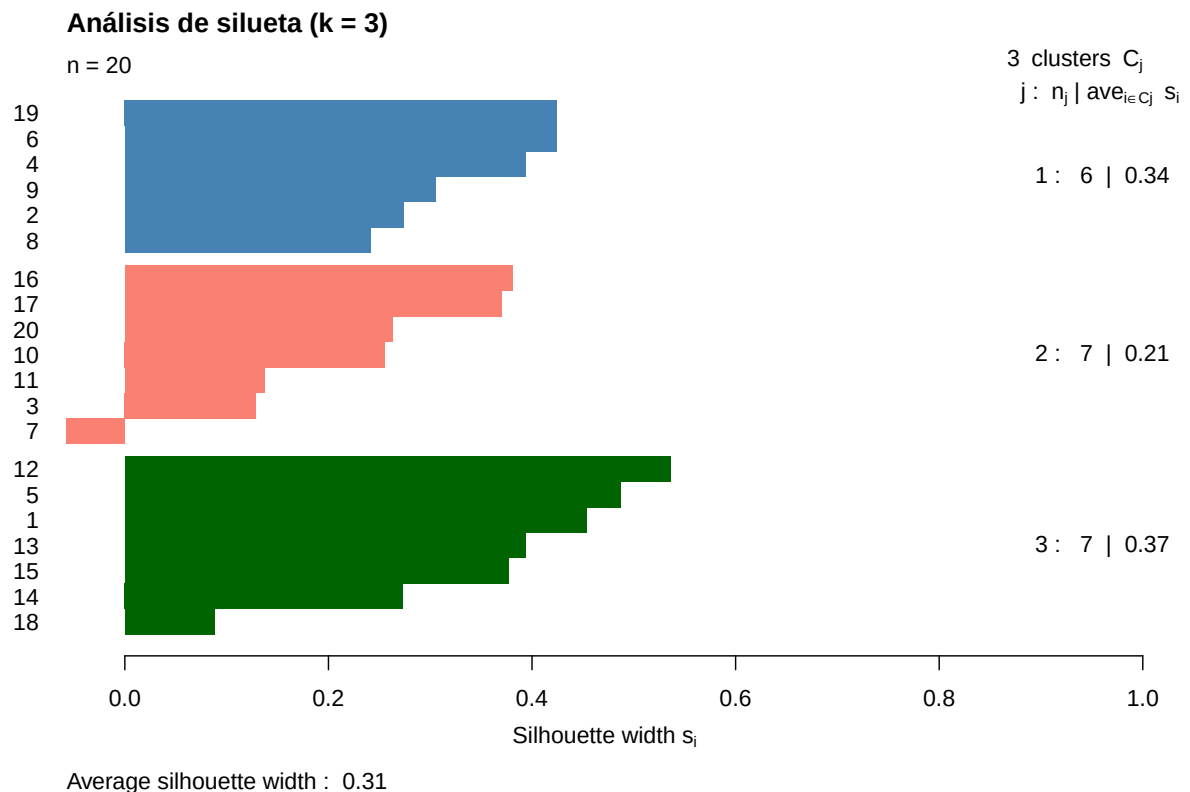
```
##      Equipo Cluster
## 1      Arsenal      3
## 2    Aston Villa      1
## 3    Bournemouth      2
## 4      Brentford      1
## 5      Brighton      3
## 6      Chelsea      1
## 7  Crystal Palace      2
## 8      Everton      1
## 9      Fulham      1
## 10     Leeds      2
## 11    Leicester      2
## 12    Liverpool      3
## 13     Man City      3
## 14   Man United      3
## 15    Newcastle      3
## 16   Nottingham      2
## 17  Southampton      2
## 18    Tottenham      3
## 19     West Ham      1
## 20     Wolves      2
```

```
# Centros de los clusters (en unidades estandarizadas)
round(kmeans_equipos$centers, 2)
```

```
##      GolesFavor GolesContra Tiros TirosPuerta Corners Faltas
```

```
## 1      -0.43      -0.28 -0.46      -0.41      -0.22      -0.61
## 2      -0.76       0.96 -0.83      -0.89      -0.79       0.78
## 3       1.13      -0.72  1.22       1.25       0.98      -0.26
```

```
# Análisis de silueta
sil <- silhouette(kmeans_equipos$cluster, dist(perfil_escalado))
plot(sil, main = "Análisis de silueta (k = 3)",
     col = c("steelblue", "salmon", "darkgreen"))
```

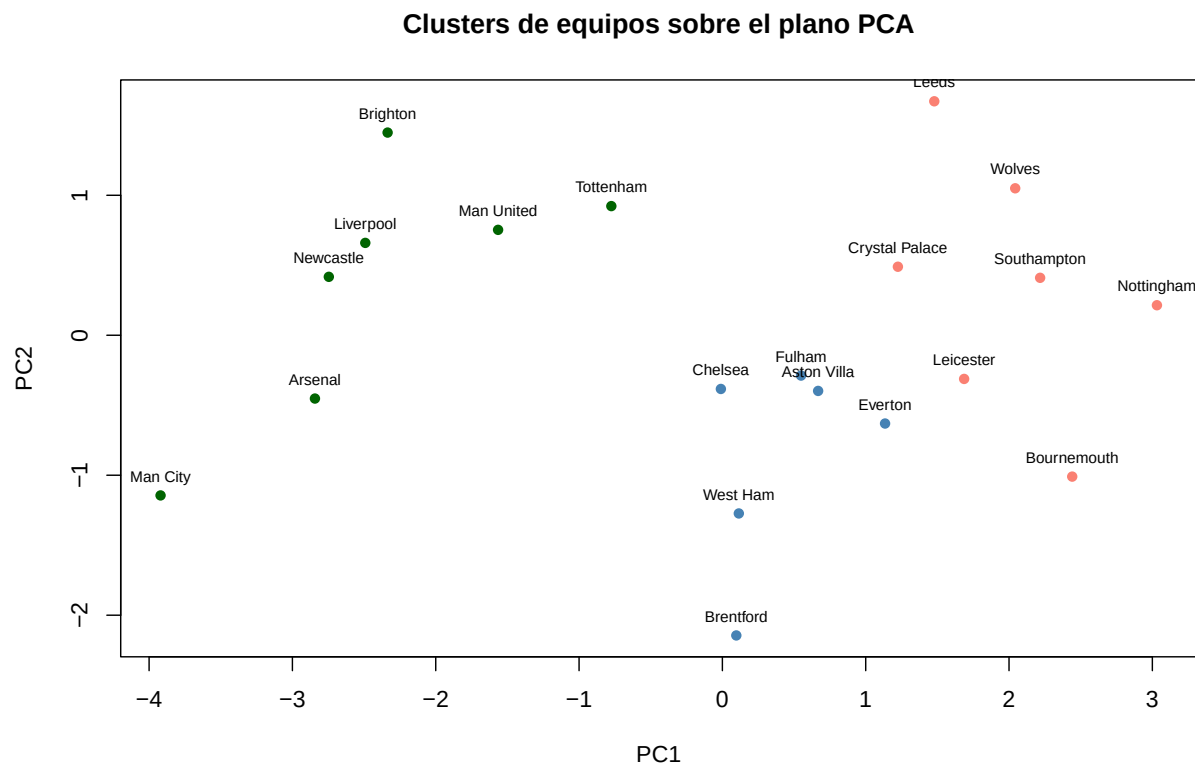


```
mean(sil[, 3])
```

```
## [1] 0.3074029
```

8.3. Interpretación táctica de los clusters

```
# Visualización de los clusters sobre el plano de las dos primeras componentes
plot(pca_equipos$x[, 1], pca_equipos$x[, 2],
     main = "Clusters de equipos sobre el plano PCA",
     xlab = "PC1", ylab = "PC2", pch = 16,
     col = c("steelblue", "salmon", "darkgreen")[kmeans_equipos$cluster])
text(pca_equipos$x[, 1], pca_equipos$x[, 2],
     labels = perfil_equipos$Equipo, cex = 0.7, pos = 3)
```



Los centros de los clusters dibujan tres perfiles tácticos nítidos. El grupo de élite reúne a los grandes de la liga (Arsenal, Liverpool, Man City, Man United y Tottenham) junto a Newcastle y Brighton: tiros, tiros a puerta y goles a favor más de una desviación típica por encima de la media, y la menor concesión defensiva (el Chelsea, en cambio, queda descolgado en la zona media, coherente con su irregular temporada). El segundo agrupa a los equipos en dificultades, incluidos los tres descendidos (Leicester, Leeds y Southampton) junto a Wolves, Nottingham, Bournemouth y Crystal Palace, con baja producción ofensiva y la mayor concesión de goles. El tercero corresponde a la zona media (Brentford, Fulham, Aston Villa, West Ham, Everton y un Chelsea muy alejado de la élite, coherente con su temporada), con valores próximos a la media en todas las facetas. El coeficiente de silueta medio de 0.307 indica una estructura de grupos real pero con fronteras difusas, lo esperable en una liga donde la zona media es competitivamente continua. Esta segmentación, obtenida sin utilizar la clasificación final, reproduce de forma notable la estructura competitiva real de la Premier League y es coherente con los parámetros de ataque y defensa estimados por el modelo Dixon-Coles en la sección 5.

9. ¿Cómo optimizar el rendimiento de un equipo?

En esta sección se plantea y resuelve un problema de programación lineal en el contexto de la planificación deportiva, aplicando las técnicas de formulación, resolución, dualidad y análisis de sensibilidad estudiadas en Optimización Matemática.

9.1. Planteamiento del problema y función objetivo

Un cuerpo técnico dispone de un máximo de 100 horas semanales de entrenamiento para repartir entre trabajo ofensivo (x_1) y trabajo defensivo (x_2). Como supuestos de planificación, se asume que cada hora

de entrenamiento ofensivo aporta 0.05 unidades de mejora al rendimiento global del equipo y cada hora defensiva 0.03 unidades. El club exige un mínimo de 30 horas defensivas (solidez competitiva) y de 20 horas ofensivas (mantenimiento de la capacidad goleadora). El problema queda formulado como:

Maximizar $Z = 0.05 \cdot x_1 + 0.03 \cdot x_2$

sujeto a:

$x_1 + x_2 \leq 100$, $x_1 \geq 20$, $x_2 \geq 30$, $x_1, x_2 \geq 0$

9.2. Restricciones y resolución del modelo

(lpSolveAPI ya cargado en 2.1)

```
# Modelo con 2 variables de decisión
modelo_lp <- make.lp(0, 2)
invisible(lp.control(modelo_lp, sense = "max")) # maximización (sin volcar ajustes)

# Función objetivo
set.objfn(modelo_lp, c(0.05, 0.03))

# Restricciones
add.constraint(modelo_lp, c(1, 1), "<=", 100) # horas totales
add.constraint(modelo_lp, c(1, 0), ">=", 20)  # mínimo ofensivo
add.constraint(modelo_lp, c(0, 1), ">=", 30)  # mínimo defensivo

# Nombres para legibilidad
dimnames(modelo_lp) <- list(c("HorasTotales", "MinAtaque", "MinDefensa"),
                             c("x1_Ataque", "x2_Defensa"))

# Resolución
solve(modelo_lp)
```

```
## [1] 0
```

```
# Valor óptimo y solución
get.objective(modelo_lp)
```

```
## [1] 4.4
```

```
get.variables(modelo_lp)
```

```
## [1] 70 30
```

La solución óptima asigna 70 horas al trabajo ofensivo y 30 al defensivo, con un valor objetivo $Z = 4.4$. La lectura es directa: dado que cada hora ofensiva rinde más que cada hora defensiva (0.05 frente a 0.03), el óptimo consiste en cubrir el mínimo defensivo exigido y dedicar todas las horas restantes al ataque.

9.3. Análisis de sensibilidad

```
# Precios sombra (valores duales de las restricciones)
get.dual.solution(modelo_lp)
```

```
## [1] 1.00 0.05 0.00 -0.02 0.00 0.00
```

```
# Rangos de sensibilidad de los coeficientes y del lado derecho
get.sensitivity.objex(modelo_lp)
```

```
## $objfrom
## [1] 3e-02 -1e+30
##
## $objtill
## [1] 1e+30 5e-02
##
## $objfromvalue
## [1] -1e+30 -1e+30
##
## $objtillvalue
## [1] NA NA
```

```
get.sensitivity.rhs(modelo_lp)
```

```
## $duals
## [1] 0.05 0.00 -0.02 0.00 0.00
##
## $dualsfrom
## [1] 5.000000e+01 -1.000000e+30 -3.552714e-15 -1.000000e+30 -1.000000e+30
##
## $dualstill
## [1] 1e+30 1e+30 8e+01 1e+30 1e+30
```

El precio sombra de la restricción de horas totales es 0.05: cada hora adicional de entrenamiento disponible incrementaría el rendimiento global en 0.05 unidades, porque esa hora extra se destinaría íntegramente al ataque. La restricción del mínimo defensivo tiene un precio sombra de -0.02: cada hora defensiva exigida por encima del óptimo libre cuesta 0.02 unidades de rendimiento (la diferencia 0.05 - 0.03 entre dedicarla al ataque o a la defensa). La restricción del mínimo ofensivo no está activa (la solución la supera con holgura), por lo que su precio sombra es cero.

9.4. Interpretación de resultados

El valor $Z = 4.4$ representa el rendimiento máximo alcanzable con los recursos y exigencias actuales. En términos prescriptivos, el modelo indica al cuerpo técnico que la restricción verdaderamente limitante es el total de horas disponibles: si se quiere mejorar el rendimiento, es más rentable ampliar la capacidad de entrenamiento que renegociar los mínimos por área. Asimismo, cuantifica el coste de oportunidad de las exigencias defensivas: cada hora defensiva obligatoria por encima del mínimo necesario reduce el rendimiento en 0.02 unidades, información directamente utilizable en la negociación entre dirección deportiva y cuerpo técnico.

10. Conclusiones

10.1. Principales hallazgos

El análisis confirma con rigor estadístico que la ventaja de jugar en casa es real y cuantificable: el modelo Dixon-Coles la estima en un factor multiplicativo de 1.34 sobre los goles esperados (+34%), en línea con los contrastes de hipótesis de la sección 4 (p-valores de $6.654\text{e-}06$ en goles y $2.737\text{e-}09$ en tiros a puerta).

El segundo hallazgo central es que la calidad de las ocasiones domina sobre el volumen de juego. La correlación de los goles con los tiros a puerta (0.59 local, 0.54 visitante) prácticamente duplica la correlación con los tiros totales, y el modelo de Poisson lo confirma: cada tiro a puerta adicional multiplica los goles esperados por 1.22, mientras que el volumen bruto de disparos no resulta significativo. La implicación táctica es directa: entrenar la selección del disparo es más rentable que acumular llegadas.

La simulación Monte Carlo separa calidad de fortuna con precisión: el título de Man City respondió a una superioridad real (campeón en el 71.6% de las 10.000 temporadas simuladas), mientras que Wolves conservó la categoría con un 45.6% de probabilidad de descenso y Leicester descendió pese a un riesgo estimado de solo el 9.2%, dos de los desajustes entre calidad y resultado más claros de la temporada.

En clasificación supervisada, la regresión logística y el LDA alcanzan un 77.6% de accuracy sobre el conjunto de prueba (en torno al 70% bajo validación cruzada), un resultado sólido para un deporte con tanto azar como el fútbol, y la segmentación no supervisada reproduce, sin conocer la clasificación, la estructura competitiva de la liga en tres perfiles tácticos. Finalmente, el modelo de optimización ilustra cómo traducir coeficientes de rendimiento en decisiones de planificación: la restricción limitante es la capacidad total de entrenamiento, y cada hora defensiva exigida por encima del óptimo tiene un coste de oportunidad medible.

10.2. Limitaciones

El estudio se apoya en una única temporada, por lo que los parámetros estimados no capturan dinámicas de forma ni cambios de plantilla entre campañas. El modelo Dixon-Coles implementado asume independencia entre los goles de ambos equipos, sin la corrección para marcadores bajos del modelo original. Los modelos de clasificación emplean estadísticas del propio partido, por lo que su uso es explicativo y no anticipatorio: predecir antes del pitido inicial exigiría covariables previas al encuentro. Por último, el dataset carece de métricas modernas como los goles esperados (xG) o datos de posesión, que enriquecerían la medición de la calidad ofensiva.

10.3. Líneas futuras

Tres extensiones naturales del trabajo: incorporar la corrección rho del modelo Dixon-Coles original para marcadores bajos y explorar la estimación bayesiana de los parámetros con distribuciones a priori informativas; aplicar modelos de ensemble (Random Forest, XGBoost) incorporando temporadas anteriores para capturar tendencias intertemporales; y extender el análisis con datos de xG y otras ligas europeas para validar si los patrones identificados son específicos de la Premier League o estructurales del fútbol de élite.